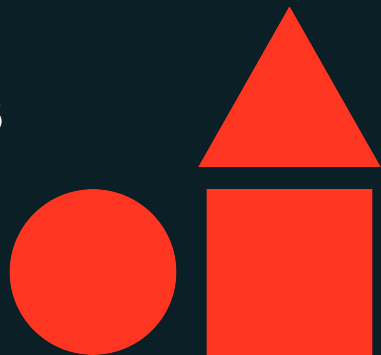




Automated Deployment with Declarative Automation Bundles

Databricks Academy



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Agenda

Course Name
• DevOps and CI/CD Review
• Deployment with Declarative Automation Bundles (DABs)
• Doing More with Declarative Automation Bundles



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Copy over this table if necessary.

Course Learning Objectives

- Review the core principles of DevOps, including continuous integration and continuous deployment (CI/CD) and testing, and how they apply to Databricks projects.
- Evaluate the tools available for continuous deployment (CD) of Databricks projects, including the Databricks REST API, SDK, and CLI.
- Explore the benefits of Declarative Automation Bundles for continuous deployment and their folder structure and main components.
- Learn how to validate, deploy, and run Declarative Automation Bundles using the Databricks CLI to multiple target environments using different configurations.
- At a high level, learn how to automate deployment pipelines for Databricks projects using GitHub Actions with Declarative Automation Bundles to streamline the CI/CD process.
- Get a quick introduction into how to use Visual Studio Code for developing, testing, and deploying Declarative Automation Bundles.



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

In this course we'll be reviewing the core principles of DevOps, continuous integration and continuous deployment (CI/CD) processes, and how they relate to Databricks projects.

Then we will focus on continuous deployment of Databricks projects, starting with reviewing available tools such as the Databricks REST API, SDK, and CLI. Then, we'll dive into the best way forward for deployment using Declarative Automation Bundles, exploring their structure and key components, and the benefits they offer for continuous deployment.

You'll learn how to validate, deploy, and run Declarative Automation Bundles using the Databricks CLI across different environments with varying configurations.

Then we'll introduce Visual Studio Code (VSCode) as a tool for developing, testing, and deploying Declarative Automation Bundles, making it easier to manage your Databricks deployment locally.

Finally, we will take a brief look at how to automate the CI/CD process using GitHub Actions with Declarative Automation Bundles for streamlined deployment pipelines.

Course Prerequisites (REQUIRED)



Proficient Knowledge of the Databricks Platform

- Databricks Workspaces
- Apache Spark
- Delta Lake and the Medallion Architecture
- Unity Catalog
- Spark Declarative Pipelines(SDP)
- Workflows
- Git Folders



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Before we start, make sure you're comfortable with a few key concepts.

First, you'll need a good grasp of the Databricks platform, including Databricks Workspaces for managing projects. Familiarity with Apache Spark and Delta Lake, especially in the context of the Medallion Architecture, is important.

You should also know how to use the Unity Catalog for data governance, and Spark Declarative Pipelines for building pipelines.

Finally, knowledge of Databricks Workflows and basic knowledge of Databricks Git Folders will help you build your jobs and manage version control.

Course Prerequisites (REQUIRED)



Proficient Knowledge of the Databricks Platform

- Databricks Workspaces
- Apache Spark
- Delta Lake and the Medallion Architecture
- Unity Catalog
- Spark Declarative Pipelines(SDP)
- Workflows
- Git Folders



Experience Ingesting and Transforming Data

- Familiarity with PySpark for data processing and DataFrame manipulations.
- Experience in writing intermediate-level queries using SQL
- Proficient in writing Python, including functions and classes.



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

You should have experience ingesting and transforming data, with a solid understanding of PySpark for data processing and DataFrame manipulation, and some experience writing intermediate-level SQL queries.

You should also be proficient in writing Python code, including functions and classes.

Course Prerequisites (REQUIRED)



Proficient Knowledge of the Databricks Platform

- Databricks Workspaces
- Apache Spark
- Delta Lake and the Medallion Architecture
- Unity Catalog
- Spark Declarative Pipelines(SDP)
- Workflows
- Git Folders



Experience Ingesting and Transforming Data

- Familiarity with PySpark for data processing and DataFrame manipulations.
- Experience in writing intermediate-level queries using SQL
- Proficient in writing Python, including functions and classes.



Knowledge of DevOps and CI/CD Principles

- Familiarity with DevOps practices and the principles behind continuous integration and continuous delivery/deployment (CI/CD).
- Knowledge of Git version control.
- DevOps Essentials for Data Engineering Course



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Lastly you should already have some familiarity with DevOps practices and the principles behind CI/CD (continuous integration and continuous delivery/deployment). Basic knowledge of Git will also be helpful.

If you need some DevOps essential knowledge, you take the DevOps Essentials for Data Engineering course.

Lab Exercise Environment

Technical Details



- Your lab environment is provided by Vocareum.
- It will open in a new tab.
- It has been configured with the permissions and resources required to accomplish the tasks outlined in the lab exercise.
- Third party cookies must be enabled in your browser for Vocareum's user experience to work properly.
- Make sure to enable pop ups!



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

As mentioned earlier, if you have access to our lab environment today, you'll be able to follow along with the demonstrations and at the end of the course be able to complete a comprehensive lab to practice what you've learned. Here's just a few notes about our lab environment.

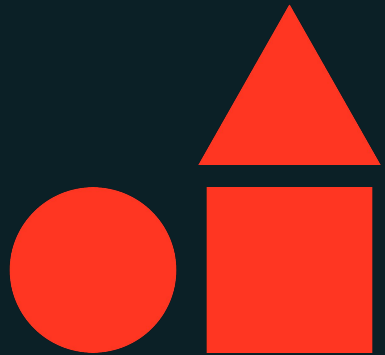
Databricks Academy labs are provided by Vocareum and when you select the Lab object in Databricks Academy, it will open into a new window. From this workspace, you'll be able to complete all the tasks outlined in the lab exercises and demos for this course.

So let's get started!



DevOps and CI/CD Review

Automated Deployment with Declarative Automation Bundles



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Agenda

DevOps and CI/CD Review

- DevOps Review
- Continuous Integration and Continuous Deployment/Delivery (CI/CD) Review
- Course Setup and Authentication



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Copy over this table if necessary.

Section Learning Objectives

- Review the core concepts and benefits of CI/CD in modern software development, with an emphasis on automation, continuous integration, and continuous delivery of data pipelines.
- Evaluate tools for continuous deployment of Databricks projects, including the Databricks REST API, SDK, and CLI, and understand their use in automating deployment processes.



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

In this section, we'll start with a quick review of DevOps and CI/CD concepts.

Then we will take a look at tools for continuous deployment in Databricks, like the REST API, SDK, and CLI, which automate the deployment of notebooks, jobs, and clusters.



DevOps and CI/CD Review

LECTURE

DevOps Review



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

DevOps Review

What is DevOps?

DevOps



Fosters **collaboration** between development and operations teams to **automate workflows** and **streamline processes**.



Key benefits include

- faster deployments
- improved collaboration
- enhanced reliability
- better scalability



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Let's quickly review what is DevOps.

DevOps is a culture and set of practices that combines Software Engineering Best Practices with IT Operations to deliver software more rapidly, efficiently, and with higher quality.

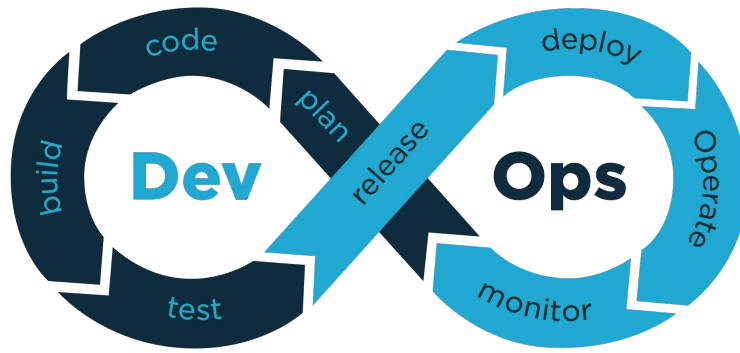
<CLICK> It's all about fostering collaboration between development and operations teams to automate your workflows, streamline the processes, and ensure continuous delivery of applications.

<CLICK> Key benefits of DevOps include:

- Faster deployment cycles
- Improved collaboration between teams
- Enhanced system reliability
- Better scalability and efficiency

In short, DevOps is a way to build and deliver software quickly and reliably by bridging the gap between development and operations.

DevOps is a process for **continuously** integrating, testing and deploying your code.



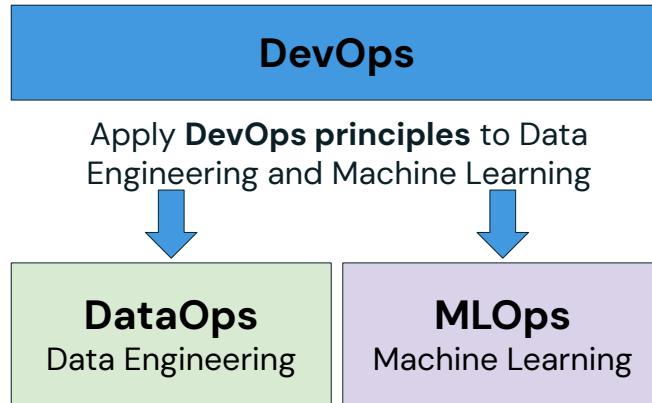
© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

The DevOps lifecycle is all about seamless collaboration between development and operations teams to deliver high-quality software.

It start with planning, coding, building and testing. Then moving to releasing, deploying, operating and monitoring.

Continuously iterating throughout the DevOps lifecycle as the project needs fixes, updates or features.

DevOps For Data Engineering and Machine Learning



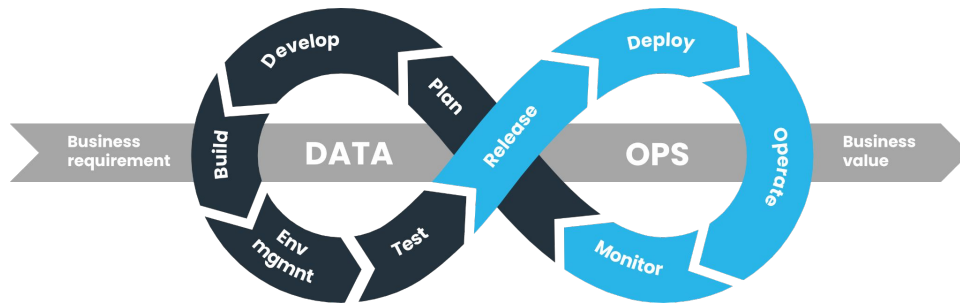
© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

We can apply DevOps principles to Data Engineering and Machine Learning. Remember, DevOps is all about automating processes, improving collaboration, testing, and speeding up delivery.

DataOps is a subset of DevOps and applies DevOps to data engineering. It automates the management of data pipelines, ensuring smooth, reliable data flows from collection to processing. This means fewer bottlenecks and faster insights.

MLOps is about applying DevOps to machine learning. It streamlines the process of deploying and managing ML models, ensuring that models move from development to production quickly and are monitored for performance.

DataOps = DevOps for Data Engineering



You want to think about how **DevOps** principles and culture can be applied to your **Data Engineering pipelines**.



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

DataOps is essentially DevOps for data engineering—it applies those same principles of automation, collaboration, and continuous improvement to data workflows. Just as DevOps streamlines software delivery, DataOps focuses on optimizing the flow, quality, and management of data pipelines.

In the end, you want to think about how DevOps principles and culture can be applied to your Data Engineering pipelines.



DevOps and CI/CD Review

LECTURE

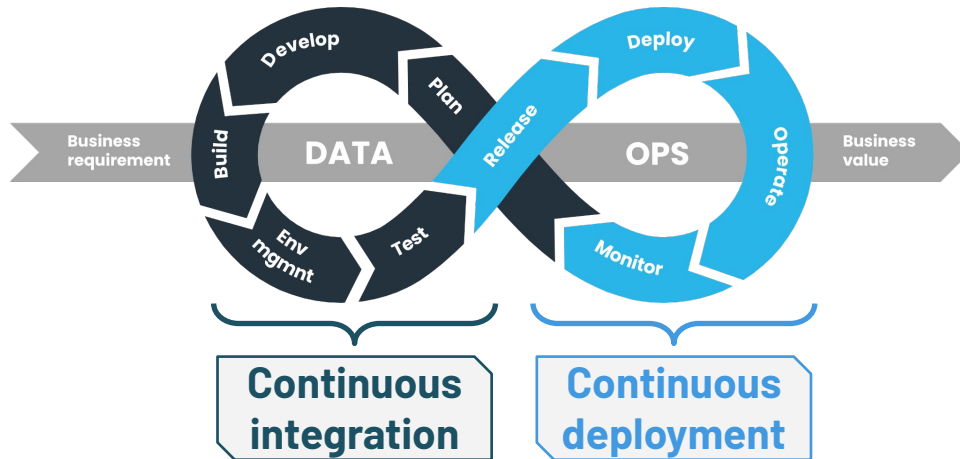
Continuous Integration and Continuous Deployment/Delivery (CI/CD) Review



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Review the Role of CI/CD in DevOps

Continuous Integration (CI) and Continuous Deployment/Delivery (CD)



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Let's do a quick high level review of the role of CI/CD in DevOps.

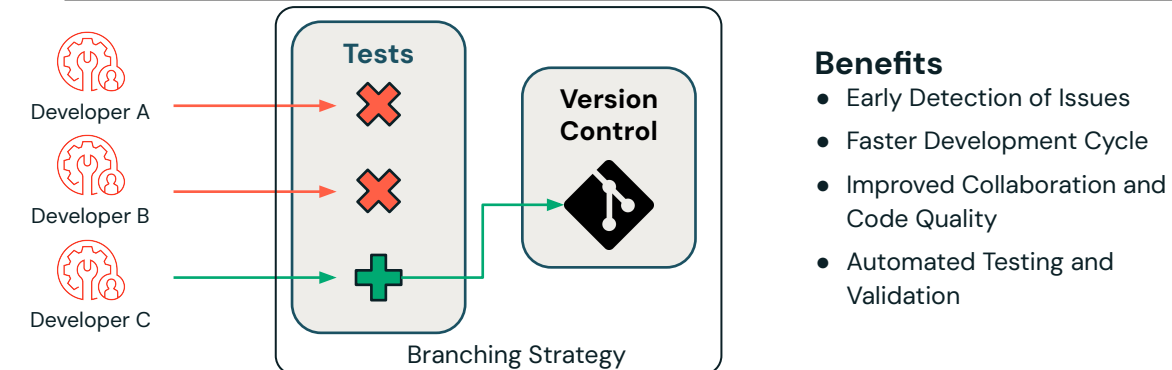
CI/CD is a key subset of DevOps practices that focuses on automating code integration, testing, and delivery. Within the DevOps lifecycle, continuous integration (CI) emphasizes planning, development, environment management, and testing of the pipelines.

On the other hand, continuous deployment/delivery (CD) focuses on automating release processes, deployment, operation, and monitoring of these pipelines.

Review the Role of CI/CD in DevOps

Continuous Integration (CI) High Level Overview

CI involves regularly **merging code** changes from **multiple contributors** into a **central repository** and running **automated tests** to ensure code quality.



Benefits

- Early Detection of Issues
- Faster Development Cycle
- Improved Collaboration and Code Quality
- Automated Testing and Validation



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

CI involves regularly merging code changes from multiple contributors into a central repository and running automated tests to ensure code quality. Tests that fail do not make it into your source code.

For example, imagine three developers working on a project, implementing updates or fixes.

<CLICK> The developers push their code to a version control system, which automatically runs tests before the commit is made. This ensures that any issues with the code are caught. If the tests fail, the commit is stopped. If the tests pass, the changes are implemented.

The way testing and commits to version control are handled depends on the branching strategy defined by your organization. Deciding on the right branching and testing strategy is crucial for maintaining quality and smooth workflows.

<CLICK> Here are four key benefits of Continuous Integration:

First, Early Detection of Issues—by integrating code often, bugs and conflicts are caught early, making them easier to fix.

Second, Faster Development Cycle—frequent integration speeds up the delivery of new features and fixes.

Third, Improved Collaboration and Code Quality—regular integration leads to cleaner, more modular code and better teamwork.

Finally, Automated Testing and Validation—automated tests run with each integration to ensure the code is stable and works with existing features.

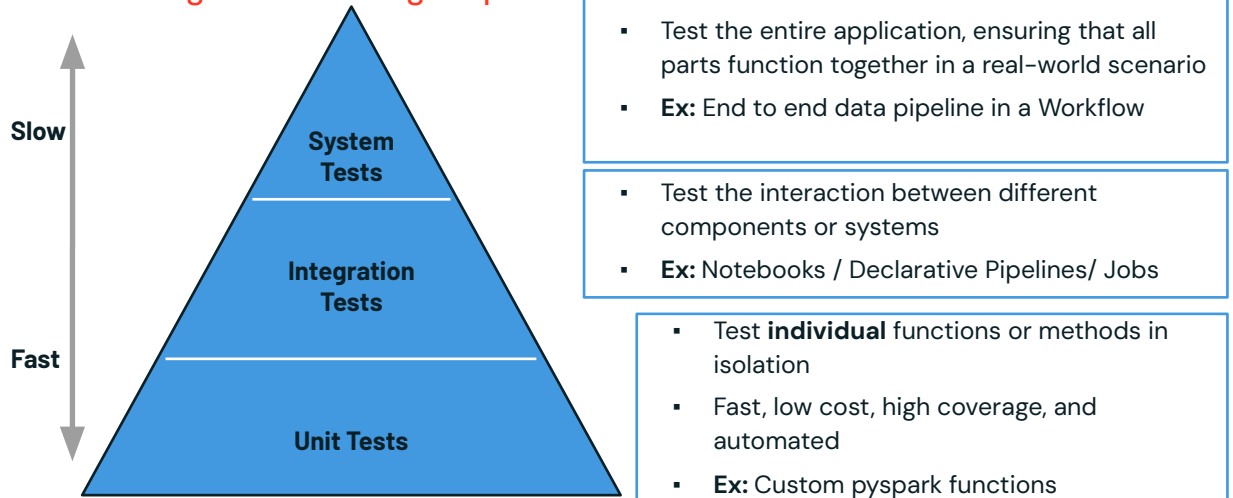
These benefits—early issue detection, faster delivery, better collaboration, and automated testing—make Continuous Integration essential for smooth development.

Slide 6: Continuous Integration (CI)

- **Title:** *What is Continuous Integration (CI)?*
- **Content:**
 - **Definition:** CI involves regularly merging code changes from multiple contributors into a central repository and running automated tests to ensure code quality.
 - **Key Practices:**
 - Frequent commits (several times a day)
 - Automated build and test on each commit
 - Fast feedback on errors
 - **Visual:** A flowchart of CI process (developer commit → build server → test → feedback loop).

Review the Role of CI/CD in DevOps

Review High-level Testing Steps



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://apache.org/).

Let's quickly review the high-level testing steps within CI/CD.

The testing steps you should following are within what's called the testing pyramid. The testing pyramid categorises different tests, unit tests, integration tests and system tests. Testing your code is extremely important.

<CLICK> The base of the pyramid are unit tests which test individual functions or methods in isolation. Since they small individual functions, they typically can run quickly, frequently and automatically, ensuring that the functions work as expected. Unit tests form the foundation because they are inexpensive and provide the broadest coverage. For example, testing if a pyspark method works as expected.

<CLICK> Next is integration tests test the interaction between different components or systems. These are typically slower and more costly than unit tests, but provide greater assurance that components work together correctly. Within Databricks these typically will revolve around using Notebooks, Lakeflow Declarative Pipeline and or Lakeflow Jobs. For example, testing whether a pyspark method and Declarative pipeline work correctly.

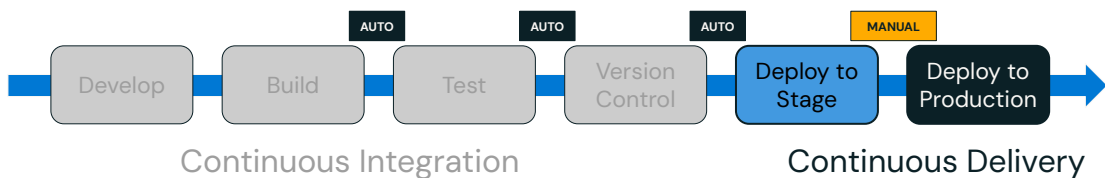
<CLICK> Lastly system tests test the entire application, ensuring that all parts function together in a real-world scenario. These are typically slow, costly, and often run in a production-like environment. For example, for our end to end data pipeline, testing whether the data data pipeline works as expected within a Workflow, creating our desired results.

Review the Role of CI/CD in DevOps

Continuous Delivery/Deployment (CD) Overview

Continuous Delivery (CD):

Automatically pushing changes to staging/pre-production environments **with the ability to manually deploy to production** at any time.



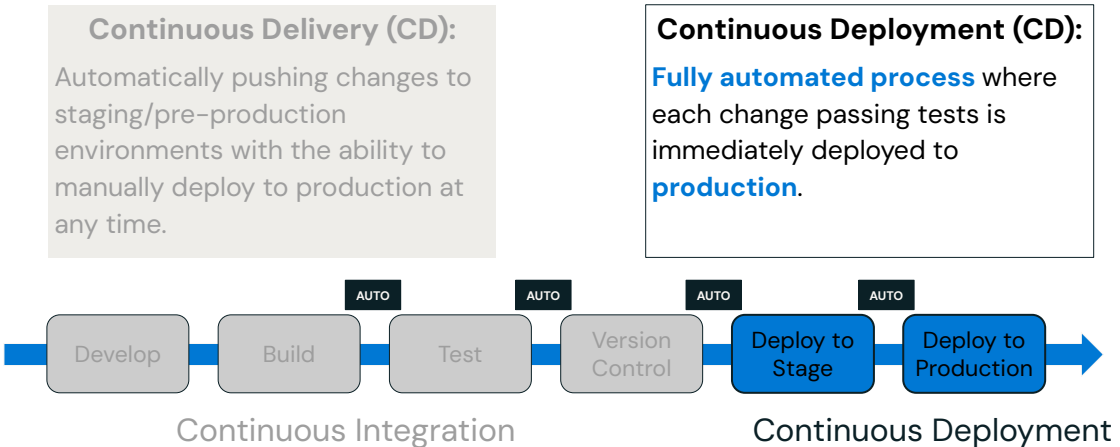
© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

The Continuous Delivery/Deployment (CD) part of is extremely important in CI/CD.

Continuous Delivery is all about automating the process of pushing changes to staging or pre-production environments. This setup allows for seamless updates and provides the flexibility to manually deploy to production whenever needed, ensuring smooth, controlled releases. For example, after the continuous integration steps are complete and testing has occurred we can decide to deploy our data pipeline to production.

Review the Role of CI/CD in DevOps

Continuous Delivery/Deployment (CD) Overview



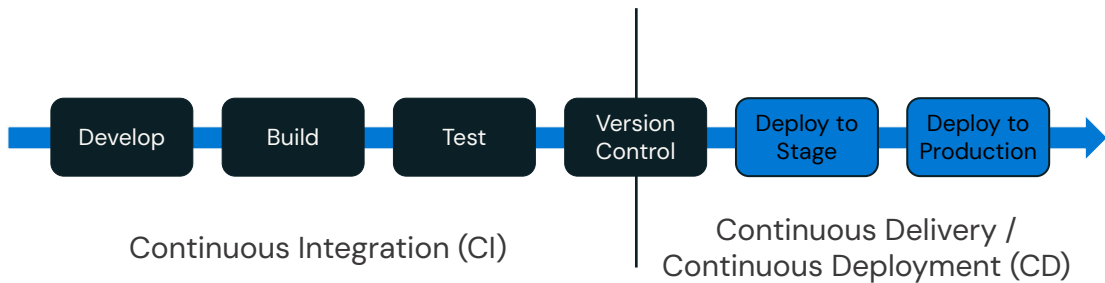
© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Continuous Deployment takes automation a step further. Once a change passes all tests, it's automatically deployed to production, ensuring that new features or fixes are delivered quickly and seamlessly, without manual intervention.

For example, if your continuous integration processes are aligned and well implemented, we can automatically deploy our data pipeline from development, to staging, then production, avoiding manual deployment steps. Implementing this technique required well thought out tests to ensure the pipeline should be deployed.

Review the Role of CI/CD in DevOps

High-Level CI/CD Workflow Overview



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Looking at the high level overview of CI/CD we want to combine continuous integration (develop, build, test and version control) with continuous delivery/deployment (deploying the project to stage and production). Automating this entire process to develop quickly and error free.

In the end, the CI/CD process streamlines development of your data pipelines by automating testing and deployment, leading to faster, more reliable releases.

This approach minimizes manual errors, improves collaboration, and ensures high-quality software delivered quickly and consistently.

Isolating Environments for CI/CD



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://apache.org/).

Isolating your environments for the different stages of development is key to developing a CI/CD pipeline. This ensures that code is developed and tested in the developing and staging prior to touching the production environment.

The minimal setup is to have two environments “Development & Stage” and “Production”, but this can vary depending on your organizations requirements.

Setting Up Your Data for CI/CD



Dev Data

- Often a Small Static Subset of Production Data
- Can be Anonymized or Synthetic Datasets
- Supports Rapid Development and Testing
- Ensures Privacy and Data Integrity



Stage Data

- Staging Data Mirrors Production Structure & Volume, Typically Static
- Can be Anonymized or Scrubbed Sensitive Information
- Ensures Realistic Testing and Validation



Prod Data

- Production Data: Live & Fully Operational
- Contains Real User Data
- Continuously Updated
- Requires High Security, Privacy, & Compliance Standards



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Within a CI/CD pipeline

<CLICK> In development, data is often anonymized or generated using synthetic datasets to allow rapid development and testing without compromising privacy or production data integrity.

<CLICK> If you have an staging environment, staging data should closely mirror production in structure and volume, with anonymized or scrubbed sensitive information to ensure realistic testing and validation.

<CLICK> Production data is live, fully operational, and continuously updated, containing real user data, and must be handled with high security, privacy, and compliance standards.

Each environment should have data suited for its specific purpose, balancing realism, security, and compliance at every stage. How this is implemented will depend on your organization and the sensitivity of your data.

Deployment Tools in Databricks Overview

REST API

- Provides direct access to Databricks functionality through HTTP requests
- Requires **manual construction** of HTTP requests and handling responses



Databricks SDK

SDKs for:

- Python
- Java
- Go
- R



Method used in this course

Databricks CLI

- Easy-to-use interface for automation from terminal, command prompt, or bash scripts.
- Use **Declarative Automation Bundles (DABs)** within the CLI to write infrastructure as code for Databricks



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

There are several tools available to deploy your Databricks projects.

<CLICK> First, you can use the Databricks REST API. The REST API provides direct access to Databricks functionality through HTTP requests. It requires manually constructing HTTP requests and handling the returned responses.

<CLICK> Next, the Databricks SDK helps accelerate development and deployment within the Databricks Data Intelligence platform. It covers all public Databricks REST API operations and supports multiple programming languages, including Python, Java, Go, and R.

<CLICK> Another tool you can use is the Databricks CLI. The Databricks CLI offers an easy-to-use interface for automating tasks from the terminal, command prompt, or bash scripts. Additionally, the Databricks CLI allows you to use Declarative Automation Bundles (DABs) to write infrastructure as code for Databricks. In this course, we will focus on this technique.

Using the Databricks CLI

Connection and Authentication



Web Terminal

- You can use the Databricks web terminal to run **Databricks CLI** commands
- Enables you to run **shell commands** in Databricks
- Uses the **latest version** of the Databricks CLI by default
- **Authentication** based on the current user
- Must be **enabled**



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

There are a variety of ways you can use the Databricks CLI, depending on your needs.

<CLICK> First, you can use the Databricks web terminal to run Databricks CLI commands directly from the web interface.

This terminal enables you to execute shell commands within Databricks, making it easier to manage and interact with your environment.

By default, it uses the latest version of the Databricks CLI, ensuring you have access to the most up-to-date features and functionality.

Authentication is based on the current user, so you'll be automatically authenticated to run commands according to your permissions.

However, keep in mind that this feature must be enabled in your environment before you can start using the web terminal.

Using the Databricks CLI

Connection and Authentication



Web Terminal

- You can use the Databricks web terminal to run **Databricks CLI** commands
- Enables you to run **shell commands** in Databricks
- Uses the **latest version** of the Databricks CLI by default
- **Authentication** based on the current user
- Must be **enabled**



VS Code

- You can use VSCode to install and use the **Databricks CLI**
- You must **authenticate** to Databricks
- You can use the **Databricks VSCode Extension** for additional features



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

You can also install and run the Databricks CLI within an Integrated Development Environment (IDE) like VSCode.

When using VSCode, it's important to authenticate to your Databricks Workspace in order to run the CLI commands.

Additionally, VSCode provides the Databricks VSCode Extension, which offers extra features to enhance your development process.

Using the Databricks CLI

Connection and Authentication



Web Terminal

- You can use the Databricks web terminal to run **Databricks CLI** commands
- Enables you to run **shell commands** in Databricks
- Uses the **latest version** of the Databricks CLI by default
- **Authentication** based on the current user
- Must be **enabled**



VS Code

- You can use VSCode to install and use the **Databricks CLI**
- You must **authenticate** to Databricks
- You can use the **Databricks VSCode Extension** for additional features



Databricks Notebook

- You can run shell commands with **%sh** in a Databricks notebook.
- You can install and use the Databricks CLI
- You must **authenticate** with a token to utilize the CLI

Main method used in this course



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Another technique you can use is Databricks Notebooks.

Within notebooks, you can run shell commands directly using the %sh magic command.

This allows you to install and use the Databricks CLI, enabling you to interact with your Databricks environment from the command line.

To use the Databricks CLI within a notebook, you must authenticate using a token, ensuring secure access to your Databricks Workspace.

In this course, we'll use notebooks to execute Databricks CLI commands, as this method provides simplicity and ease of learning. However, please keep in mind that this approach might not be the best fit for your organization's needs. While it works well in a learning environment, you may need a different approach in production.



DevOps and CI/CD Review

DEMONSTRATION

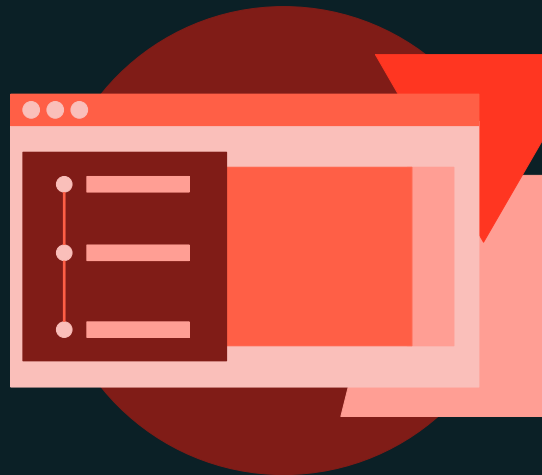
Course Setup and Authentication

You must execute the following notebook to set up your environment for the remaining demonstrations and labs.

Notebook: `./0 - REQUIRED - Course Setup and Authentication`



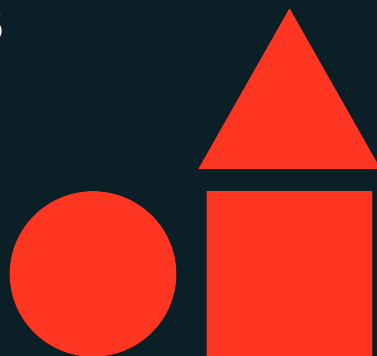
© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).





Deployment with Declarative Automation Bundles (DABs)

Automated Deployment with Declarative Automation Bundles



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Section Learning Objectives

- Discuss the components of Databricks projects and how they are integrated into a CI/CD pipeline for seamless deployment and management.
- Understand the benefits and core components of Declarative Automation Bundles (DABs).
- Learn how to validate, deploy, and execute Declarative Automation Bundles effectively.
- Explore variable substitutions and environment configuration overrides based on the target environment.
- Learn about Declarative Automation Bundles templates.
- Apply a CI/CD pipeline using a Databricks project with Declarative Automation Bundles for streamlined deployment.



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).



Deployment with Declarative Automation Bundles (DABs)

LECTURE

Deploying Databricks Projects



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Deploying Databricks Projects

Typical Databricks Projects

Consist of a variety of components

Code

Notebooks, Python .whl, JAR, dbt, etc.

Execution Environment

Databricks Workspace, compute configuration

Resources:

Databricks Workflows, MLflow Tracking Server and Registry, SDP...

Projects produce a variety of data products

- tables
- pipelines
- jobs
- machine learning models
- dashboards
- call external services
- Etc.

The deliverable determines the components

- A **simple report** might consist of a notebook running on single node compute
- A **full MLOps pipeline** would require MLflow, Feature Store, and Model Serving components



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Let's quickly recap the typical components of a Databricks project.

<CLICK> First, a Databricks project consists of multiple components, including code, notebooks, Python files, Python wheel files, JARs, DBT, and more.

It also includes an execution environment within one or more Databricks workspaces, along with specific compute configurations for processing.

Lastly, the project may involve other Databricks resources, such as Lakeflow Jobs, MLflow, Lakeflow Declarative Pipelines, and more.

<CLICK> Next, a Databricks project typically produces a variety of data products, including tables, pipelines, models, jobs, dashboards, and more.

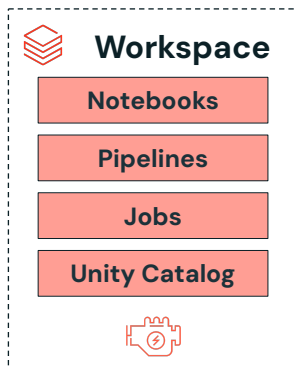
<CLICK> Lastly, the deliverables of your project determine the components you'll need.

For example, a simple report might include one or more notebooks running on a basic single-node compute.

On the other hand, a full MLOps pipeline would require multiple components, such as MLflow, the Feature Store, model serving components, notebooks, and more.

Deploying Databricks Projects

Simple Example Project Components



- **Code**
 - Python/SQL/R notebooks
- **Resources**
 - Lakeflow Jobs
 - Spark Declarative Pipelines(SDP)
- **Execution Environment**
 - Databricks Workspace
 - Unity Catalog
 - Compute configuration(s)

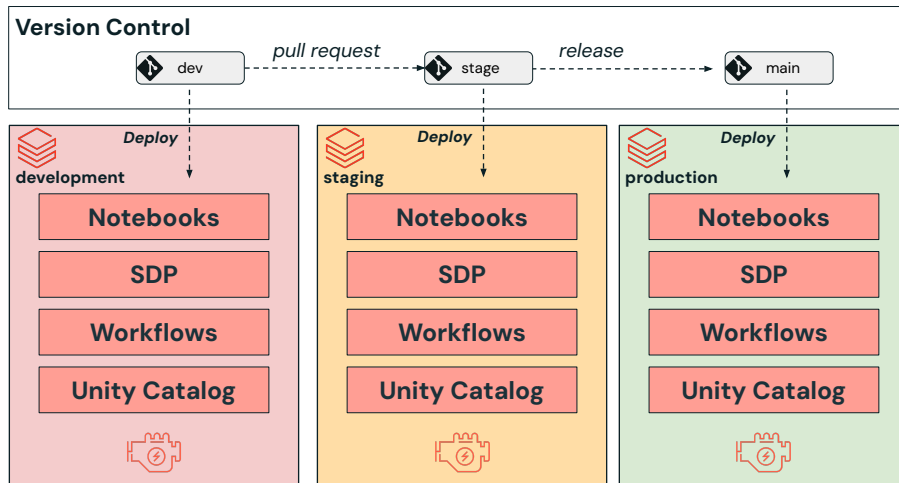


© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

For example, a simple Data Engineering Databricks project could include notebooks, Spark Declarative Pipelines, a workflow, catalogs within Unity Catalog, and specific compute configurations for the project.

Deploying Databricks Projects

CI/CD Journey to Production



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Let's take a look at a high-level CI/CD journey to production.

To start, you'll want to have a version control strategy in place. For example, your strategy might include dev, stage, and main branches.

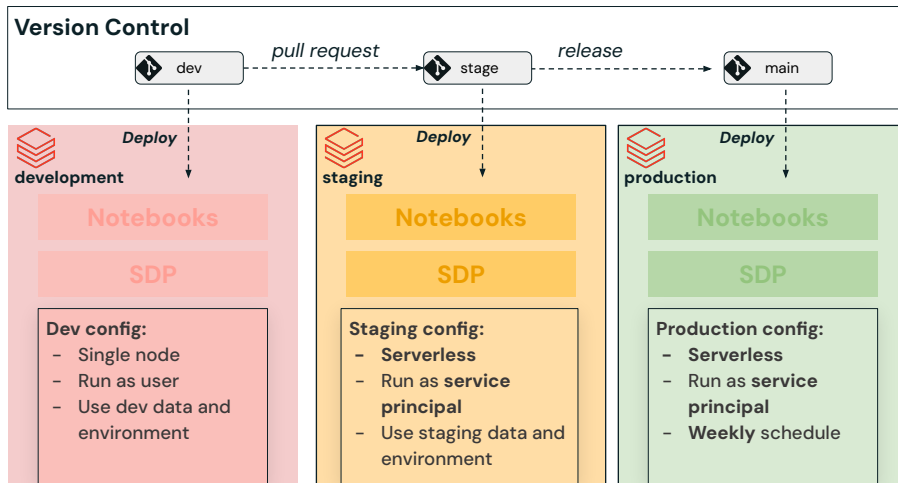
<CLICK> From here, begin by deploying your Databricks project to the development environment. This allows you to test any changes or updates made to your project on development data, using specific development configurations.

<CLICK> After completing development and creating a pull request into your stage branch, you will then deploy to the staging environment. This ensures testing of your project continues on new staging data with the appropriate staging configurations.

<CLICK> Once both your development and staging environments have completed testing and passed all tests, you can deploy to your main or production branch. This will fully deploy your project to production, utilizing production data and the necessary production configurations.

Deploying Databricks Projects

CI/CD Journey to Production



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Let's go over some example configurations for each environment.

For development, we'll configure the environment to use single-node compute, as our data is small and doesn't require a large cluster. We'll run the project using our user account on the development data.

<CLICK> In staging, we aim to closely resemble the production environment. Here, we'll use Serverless compute to scale our resources as needed. We'll run the project using a service principal identity, which gives automated tools access to only the necessary Databricks resources, offering better security than using user or group access. This will be done with our staging data.

<CLICK> Lastly, for production, we'll continue using Serverless compute and the service principal identity. The project will run using production data, and we'll set the project to run weekly to update the data for our consumers.

Deploying Databricks Projects

How do we orchestrate this CI/CD journey?

Manually

Using the UI

PROS:

- Easy to learn, high level

CONS:

- Time consuming
- Error prone
- Not a viable option for the CI/CD process

Programmatically

Databricks REST API or SDK

PROS:

- Low level control

CONS:

- Lots of APIs to have to learn
- Lots of classes to learn
- Time consuming to automate the entire CI/CD process

Terraform

Databricks Terraform provider

PROS:

- Very powerful and expressive
- Admin tool of choice for configuration

CONS:

- Can be challenging to learn for data scientists and engineers



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

So, how can we orchestrate this CI/CD journey to production?

Let's explore three different methods.

<CLICK> First, we can do this manually.

A pro of this method is that the user interface is easy to learn and navigate.

However, the cons include that it is extremely time-consuming to manually deploy the project every time. It's also highly error-prone since it involves human interaction, and, importantly, it's not a viable option for the CI/CD process where we want to automate the deployment process and easily move from one environment to the next.

<CLICK> Next, we can do this programmatically using the Databricks REST API or Databricks SDK.

A pro of this method is that it gives you low-level control, allowing you to program everything you need.

However, the cons are that this approach requires intermediate to advanced programming knowledge. You'll need to learn a lot of APIs if you're using the REST API or a lot of classes for the SDK method, and coding everything to automate the entire CI/CD process can be extremely time-consuming.

<CLICK> Lastly, for those who are experienced with Terraform, you can use that.

It's a very powerful and expressive tool that administrators typically use to manage infrastructure.

While this method might work well for some, it can be challenging for data scientists and engineers who may not have a deep background in infrastructure management. It also adds another tool that you have to learn and manage.

Deploying Databricks Projects

How Can the CI/CD Process be Simplified?

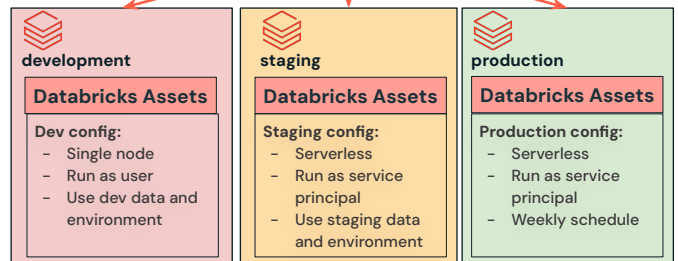
How to write code once, and then deploy to multiple environments easily?



Deploy to **assets** and **configuration** settings to multiple environments

What if we could:

- Co-version code with all configurations in a simple, easy to understand format like YAML?
- Define Databricks resources using existing REST API parameters?
- Ensure user isolation during deployment?
- Specify environment-based overrides and variables?



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

So, how can we simplify the CI/CD process for Databricks projects? How can we write code once for our project, then easily deploy it to multiple environments while modifying our configurations?

<CLICK> What if we could?

Co-version code with all configurations in a simple, easy-to-understand format like YAML? YAML is a human-readable format that organizes data in a simple, easy-to-read structure using indentation and minimal syntax. This makes it easier to understand and edit compared to other formats like XML or JSON.

<CLICK> Define Databricks resources using existing REST API parameters? This ensures we keep parameters consistent with the other techniques.

<CLICK> Ensure user isolation during deployment? This helps prevent users from interfering with each other's work during deployment.

<CLICK> Specify environment-based overrides and variables? This allows us to easily override values based on the target environment we want to deploy to, ensuring flexibility in our configuration for each environment.

Deploying Databricks Projects

Introducing Declarative Automation Bundles (DABs)!



Declarative Automation Bundles (DABs)

A tool to facilitate the adoption of software engineering best practices, including source control, code review, testing, and **continuous integration and delivery (CI/CD)**, for your data and AI projects



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Introducing Declarative Automation Bundles (or DABs)!

<CLICK> A tool to facilitate the adoption of software engineering best practices, including source control, code review, testing, and continuous integration and delivery (CI/CD), for your data and AI projects.

Moving forward, Databricks recommends DABs for creating, developing, deploying, and testing jobs and other Databricks resources as source code.



Deployment with Declarative Automation Bundles (DABs)

LECTURE

Introduction to Declarative Automation Bundles (DABs)



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Introduction to DABs

Write code once, deploy everywhere



What are Declarative Automation Bundles (DABs)?

DABs use **YAML files** to specify the artifacts, resources, and configurations of a Databricks project.



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

DABs enable us to write code once and deploy everywhere. But what exactly are they?

DABs (Declarative Automation Bundles) use YAML files to specify the artifacts, resources, and configurations of a Databricks project. They allow us to manage and deploy all the necessary components of a Databricks project in a consistent and efficient way to multiple environments.

Introduction to DABs

Write code once, deploy everywhere



What are Declarative Automation Bundles (DABs)?

DABs use **YAML files** to specify the artifacts, resources, and configurations of a Databricks project.



How do Declarative Automation Bundles work?

The new **databricks CLI** has specific bundle commands to **validate**, **deploy** and **run** Declarative Automation Bundles using a bundle YAML file



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

How exactly do DABs work?

The new Databricks CLI has specific bundle commands to validate, deploy, and run Declarative Automation Bundles using a YAML file. We will learn how to do this throughout the course.

Introduction to DABs

Write code once, deploy everywhere



What are Declarative Automation Bundles (DABs)?

DABs use **YAML files** to specify the artifacts, resources, and configurations of a Databricks project.



How do Declarative Automation Bundles work?

The new **databricks CLI** has specific bundle commands to **validate, deploy and run** Declarative Automation Bundles using a bundle YAML file



Where are Declarative Automation Bundles used?

Bundles are extremely useful during **development and CI/CD processes** to deploy Databricks assets to specified environments



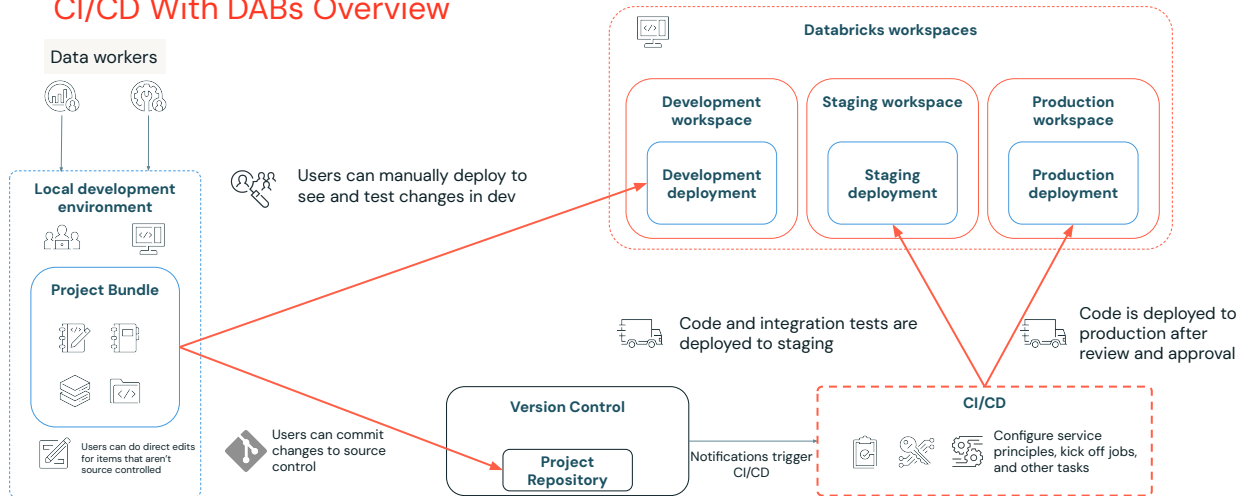
© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

So, where are Declarative Automation Bundles used?

DABs are extremely useful during development and CI/CD processes for deploying Databricks assets to target environments, while modifying specific configurations for each environment. We will see how to do this throughout the course.

Introduction to DABs

CI/CD With DABs Overview



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Let's take a look at the following diagram, which provides a high-level view of a development and CI/CD pipeline with bundles.

If you're working locally, you'll build the project bundle with your team using a local environment setup (or you can do this within your Databricks Workspace).

<CLICK> From here, you can either use bundles or manually deploy to your development environment, which can include a development workspace or catalogs. This is where you can test your changes away from production.

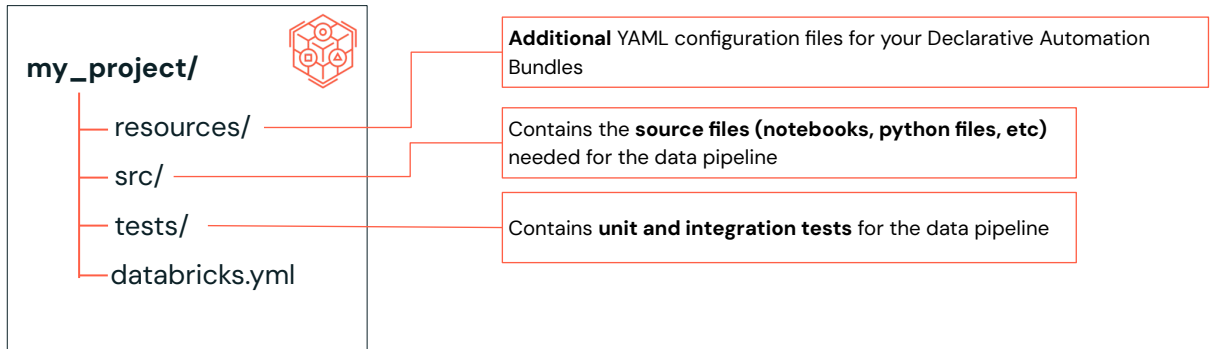
<CLICK> After development, users can then commit changes to version control within a project repository and push your development changes.

<CLICK> After the changes are committed, you can set a notification to trigger to initiate the CI/CD pipeline. In a typical pipeline, the tests are first deployed to a staging environment.

<CLICK> Once all tests pass in staging, the code can then be deployed to the production environment after following your organization's processes, such as code review and approval.

Introduction to DABs

Simple Project Structure Overview



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Now that we have a good overview of the CI/CD process, let's dive into a typical, simple project structure for your bundle.

In your project folder, you'll want to create multiple subfolders to organize your assets. A simple project structure should start with your project folder, followed by various folders and files, such as resources, src, tests, and a main databricks.yml file. These folders may contain additional subfolders to store specific files.

<CLICK> The resources folder should contain additional YAML configuration files for your Declarative Automation Bundles, if needed.

<CLICK> The src (or source) folder contains the source files needed for your data pipeline, such as notebooks and Python files.

<CLICK> The tests folder contains your unit and integration tests for the pipeline.

Organizing and modularizing your folders and files will help you during development and maintenance as your project grows.

This is a simple example, and your project may have additional files and folders, or a different organizational structure.

Introduction to DABs

Simple Project Structure Overview



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

The `databricks.yml` file is a required bundle configuration file used to deploy your Databricks assets. This file must:

- Be expressed in YAML format.
- Contain at minimum the top-level bundle mapping.
- Contain at least one (and only one) bundle configuration file named `databricks.yml`.

Top Level Mappings

The **databricks.yml** configuration top level mappings include:

- **bundle**
- **resources**
- **targets**
- variables
- workspace
- permissions
- artifacts
- include
- sync

databricks.yml file example

```
bundle:
  name: demo01_bundle

resources:
  jobs:
    ll_simple_dab:
      name: my_job_name_ll_simple_dab
      tasks:
        - task_key: create_bronze_table
          notebook_task:
            notebook_path: ./src/create_bronze_table.py
            source: WORKSPACE
      ...

  targets:
    development:
      mode: development
      default: true
      workspace:
        host: https://dev.cloud.databricks.com/

    production:
      mode: production
      workspace:
        host: https://test.cloud.databricks.com/
```



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

The databricks.yml file is key to your bundle. Let's take a look at a simple example.

First, the YAML file contains several top-level mapping keys that are left-aligned. In this example databricks.yml file, it includes the top-level mappings: bundle, resources, and targets.

<CLICK> Other top-level mappings include variables, workspace, permissions, artifacts, include, and sync. We will cover many of these top-level mappings in this course.

The **bundle** top level mapping declares a required bundle **name**, and can use other optional configurations

The top-level mapping of **resources** defines the databricks resources used by the bundle, including:

- Lakeflow Jobs
- Lakeflow Declarative pipelines
- MLflow
- And more

Resources are defined using the corresponding **Databricks REST API**.

databricks.yml file example

```
bundle:
  name: demo01_bundle

resources:
  jobs:
    l1_simple_dab:
      name: my_job_name_l1_simple_dab
      tasks:
        - task_key: create_bronze_table
          notebook_task:
            notebook_path: /src/create_bronze_table.py
            source: WORKSPACE

  targets:
    development:
      mode: development
      default: true
      workspace:
        host: https://dev.cloud.databricks.com/

    production:
      mode: production
      workspace:
        host: https://test.cloud.databricks.com/
```

Job key → Job Name

Starting December 20, 2024, the default format for new notebooks is now .ipynb format. Make sure to specify the correct notebook extension.



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

A bundle configuration file must contain only one top-level **bundle** mapping that associates the entire bundle's contents with a name. Additionally, you can include other Databricks workspace settings, such as **cluster_id**, **compute_id**, **git**, and a few others, if needed. The additional mappings beneath the top-level mapping must be indented.

The following example declares the top-level mapping key **bundle**, with a **name** mapping that specifies the bundle name, demo01_bundle.

<CLICK> The **resources** mapping specifies information about the Databricks resources used by the bundle, such as:

Lakeflow Jobs
Lakeflow Declarative pipelines
MLflow
And more

The resources are defined using the corresponding Databricks REST API parameters.

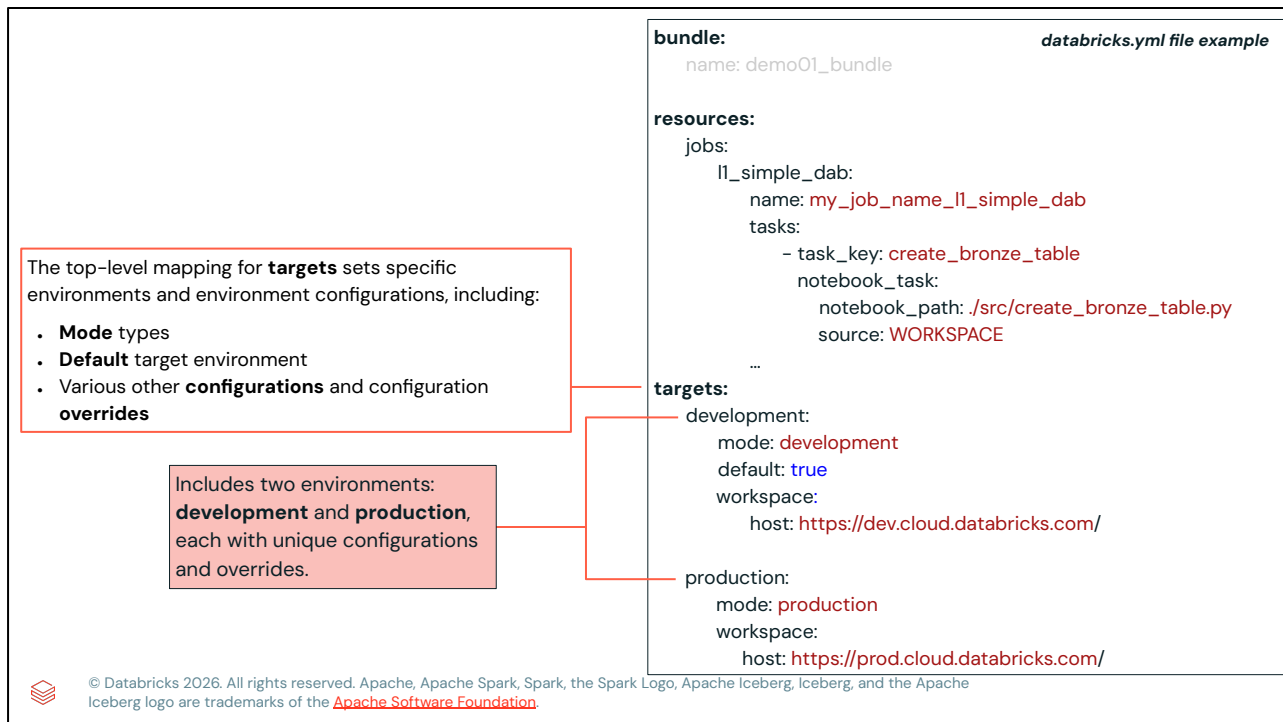
<CLICK> Now, each **resource** type mapping includes one or more individual resource declarations, each of which must have a unique name.

In this example, the job has the resource mapping **name** *l1_simple_dab*. This job creates a job named *my_job_name_l1_simple_dab* using the **name** mapping key,

and this job contains one or more tasks specified using the corresponding REST API parameters.

Within the **tasks** mapping, the task is named *create_bronze_table* and uses the **notebook_path** mapping key to specify the notebook to use. It's important to use a relative path of the notebook within your project here, with the correct extension for the notebook.

<CLICK> Be aware that starting December 20, 2024, the default format for new notebooks will be the .ipynb format. If you do not supply the correct notebook extension, an error will be returned. In this example this notebook is using the traditional.py extension.



<CLICK> The top-level mapping key targets sets specific environments and environment configurations, including:

- The environment mode type, such as development or production.
- The default target environment, which should be set to the development environment. This ensures that if you don't specify where to deploy this bundle and run, it will default to the development environment.
- In the targets mapping, you can include various other configurations and configuration overrides for that specific target.

<CLICK> In this example, we include two target environments: **development** and **production**, each with unique configurations and overrides.

In this simple example, we specify the **mode** development and **default true** mappings for the development environment, with a specific development workspace URL.

For the production environment, we use the **mode production** and specify the production workspace URL.

Validate, Deploy and Run Your DAB

Using the Databricks CLI

1

```
databricks bundle validate
```

Returns **warnings** if unknown resource properties are found in bundle configuration files.

2

```
databricks bundle deploy -t development
```

Specifies which environment to **deploy** your bundle into. In this example, the bundle will be deployed to the **development** environment.

3

```
databricks bundle run -t development  
l1_simple_dab
```

Specifies to run your bundle in the environment. You must specify the **job key name** to run the bundle job.

```
bundle:  
  name: demo01_bundle  
  
resources:  
  jobs:  
    l1_simple_dab:  
      name: my_job_name_l1_simple_dab  
      tasks:  
        - task_key: create_bronze_table  
          notebook_task:  
            notebook_path: ./src/create_bronze_table.py  
            source: WORKSPACE  
      ...  
  targets:  
    development:  
      mode: development  
      default: true  
      workspace:  
        host: https://dev.cloud.databricks.com/  
    production ...
```



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Now that we have a good understanding of the databricks.yml file, let's explore how to validate, deploy, and run our Declarative Automation Bundles.

<CLICK> First, you'll want to validate your bundle. You can do this with the Databricks CLI command **databricks bundle validate**.

<CLICK> Next, you'll deploy the bundle to your Databricks Workspace. To do this, use the **databricks bundle deploy** command, the **-t** flag for target, and specify the environment you want to deploy to. In this example, we will deploy to the **development** environment. By default, development is set as the default target, so if you don't specify where to deploy, it will go to development. However, it's best practice to be explicit.

<CLICK> Once the bundle is deployed to Databricks, you will typically want to run it. To run the job in the bundle, use the **databricks bundle run** command, the **-t** flag for development, and the key that specifies the job to run. Here the key is **l1_simple_dab**.

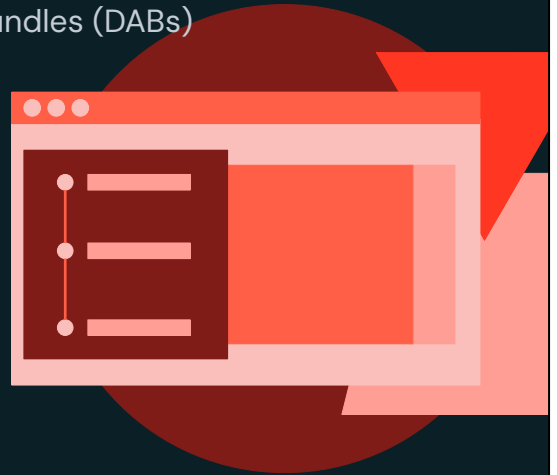
And that's it! We've covered how to create, validate, deploy, and run a simple Declarative Automation Bundle!



Deployment with Declarative Automation Bundles (DABs)

DEMONSTRATION

Deploying a Simple DAB



Notebook: /01 – Deploying a Simple DAB/Demo – Deploying a Simple DAB



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).



Deployment with Declarative Automation Bundles
(DABs)

LAB EXERCISE

Deploy a Simple DAB



Notebook: /O2L – Deploy a Simple DAB/Lab – Deploy a Simple DAB



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).



Deployment with Declarative Automation Bundles (DABs)

LECTURE

Variable Substitutions in DABs



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Variables in Declarative Automation Bundles (DABs)

Overview

Default Substitutions

By default there are a variety of **variable substitutions** available:

- `${bundle.name}`
- `${bundle.target}`
- `${workspace.file_path}`
- `${workspace.root_path}`
- `${resources.jobs.<job-name>.id}`
- `${resources.models.<model-name>.name}`
- `${resources.pipelines.<pipeline-name>.name}`



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Declarative Automation Bundles support substitutions and custom variables, making your bundle configuration files more modular and reusable. Both substitutions and custom variables allow dynamic retrieval of values, meaning that settings can be determined at the time the bundle is deployed and run.

To use a variable in your YAML file, reference it using a dollar sign, curly braces, and the mapping, then the variable name.

By default, a variety of substitutions are available. Some common ones are shown on the screen. A few to keep in mind include `bundle.target` to get the value of the target environment, `workspace.file_path` for the workspace file path, and `resources.jobs.job name` or `id`, to name a few.

Variables in Declarative Automation Bundles (DABs)

Overview

Default Substitutions

By default there are a variety of **variable substitutions** available:

- `${bundle.name}`
- `${bundle.target}`
- `${workspace.file_path}`
- `${workspace.root_path}`
- `${resources.jobs.<job-name>.id}`
- `${resources.models.<model-name>.name}`
- `${resources.pipelines.<pipeline-name>.name}`

Simple Custom Variables

- You can define **simple custom variables** in your bundle to enable dynamic retrieval of values needed for many scenarios.
- Custom variables are declared in your bundle configuration files within the **variables** mapping

variables:

```
my_lab_user_name:  
  description: Your user name  
  default: labuser23904
```



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

You can also use simple custom variables you define in your Declarative Automation Bundles.

Custom variables allow you to enable dynamic retrieval of values needed for various scenarios. These variables are declared in your bundle configuration files within the variables mapping.

For example, under the variables top-level mapping, you can specify any number of custom variables.

In this example, we'll create a variable named **my_lab_user_name**, followed by a description and its default value of *labuser23904*.

Variables in Declarative Automation Bundles (DABs)

Overview

Default Substitutions

By default there are a variety of **variable substitutions** available:

- `${bundle.name}`
- `${bundle.target}`
- `${workspace.file_path}`
- `${workspace.root_path}`
- `${resources.jobs.<job-name>.id}`
- `${resources.models.<model-name>.name}`
- `${resources.pipelines.<pipeline-name>.name}`

Simple Custom Variables

- You can define **simple custom variables** in your bundle to enable dynamic retrieval of values needed for many scenarios.
- Custom variables are declared in your bundle configuration files within the **variables** mapping

```
variables:
  my_lab_user_name:
    description: Your user name
    default: labuser23904
```

Complex Variables

- A custom variable is assumed to be of type string unless you define it as a **complex variable**.
- Define a custom variable by setting the type to **complex**

```
variables:
  my_cluster:
    description: "My cluster"
    type: complex
    default:
      spark_version: "15.4.x-scala2.11"
      node_type_id: "Standard_DS3_v2"
      num_workers: 2
```



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

You can define both simple and complex custom variables in your Declarative Automation Bundles to enable dynamic retrieval of values for various scenarios.

By default, a simple custom variable is assumed to be of type string. However, if you need more complex data, you can define a custom variable as a complex type using the **type complex** mapping.

For example, in this case, we create a complex variable named **my_cluster**, with a default value specifying the Spark version, node type, and the number of workers.

Creating Simple Custom Variables

The bundles settings file can contain one top-level **variables** mapping where custom variables are defined

Create a variable named **my_lab_user_name** with the default value of *labuser23904*

Create variables named **catalog_dev** and **catalog_prod** uses the **my_lab_user_name** value and appends *values*

catalog_dev = *labuser23904_1_dev*

catalog_prod = *labuser23904_3_prod*

databricks.yml file example

```
...
variables:
  my_lab_user_name:
    description: Your user name
    default: labuser23904

  catalog_dev:
    description: Development catalog reference
    default: ${var.my_lab_user_name}_1_dev

  catalog_prod:
    description: Production catalog reference
    default: ${var.my_lab_user_name}_3_prod
```

Reference a custom variables



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Let's look at an example of creating and referencing simple custom variables in your databricks.yml file.

First, you define the **variables** top-level mapping. Under **variables**, you can add your custom variables. The bundle settings file can only contain one top-level variables mapping, where all your custom variables are defined.

<CLICK> Create a variable named **my_lab_user_name** with a default value of *labuser23904*.

<CLICK> You can then reference the custom variable **my_lab_user_name** within other variables or mappings. In this example, we will use the **my_lab_user_name** variable to create two new variables: **catalog_dev** and **catalog_prod**, appending the strings *_1_dev* and *_3_prod* to the user name. To reference a custom variable, use the top-level mapping **var**, followed by a dot and the variable name **my_lab_user_name** within curly braces, and then append the literal string.

<CLICK> As a result, the **catalog_dev** variable will be equal to *labuser23904_1_dev*, and the **catalog_prod** variable will be equal to *labuser23904_3_prod*.

This technique allows you to reference or override variables throughout the databricks.yml file. We'll look at more examples of how to apply this.

Defining a Complex Variable

A variable is assumed to be of type string unless you define it as a **complex** variable.

databricks.yml file example

```
bundle:
  name: demo03_bundle
...
variables:
  my_cluster:
    description: "My cluster"
    type: complex
    default:
      spark_version: "15.4.x-scala2.11"
      node_type_id: "Standard_DS3_v2"
      num_workers: 2
...
...
```



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

By default, a variable is assumed to be of type string unless you define it as a complex variable.

In this example, we define the variable **my_cluster** as type complex and specify its default values. This allows you to provide more structured data for the variable, such as specifying a Spark version, node type, and number of workers.

Lookup Variables

Dynamically Retrieve an Object's Value

For specific object types you can **define a lookup** for your custom variable to retrieve the object's ID

Create the **new** variable

Lookup the cluster name to retrieve the cluster ID

```
...  
variables:  
  
  my_cluster_id:  
    description: "Get the cluster ID using a lookup variable"  
    lookup:  
      cluster: myclustername  
  
...
```



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

For specific object types, you can define a lookup for your custom variable to retrieve the object's ID using this format:

<CLICK> First, specify the variable name. In this example, the variable is called **my_cluster_id**. Then, use the lookup mapping to define what to look up. In this case, we are looking up the cluster name **myclustername** to retrieve the corresponding cluster ID value.

If a lookup is defined for a variable, the ID of the object with the specified name is used as the value of the variable. This ensures the correct resolved ID of the object is always used for the variable.

Lookup Variables

Can Define Lookups for the a Variety of Environment Configurations

- alert
- cluster_policy
- cluster
- dashboard
- instance_pool
- job
- metastore
- notification_destination
- pipeline
- query
- service_principal
- warehouse



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

You can use lookup variables for the alert, cluster_policy, cluster, dashboard, instance_pool, job, metastore, notification_destination, pipeline, query, service_principal, and warehouse object types at the time of this recording.

Target Environment Variable Overrides

Dynamically **modify variable values** within each target environment using the **target** top-level mapping.

When deploying to **development**, the **target_catalog** variable will use the **catalog_dev** value.

When deploying to **production**, the **target_catalog** variable will use the **catalog_prod** value.

```
...
targets:
  development:
    ...
    variables:
      target_catalog: ${var.catalog_dev}
  production:
    ...
    variables:
      target_catalog: ${var.catalog_prod}
```

A default value for the variable **must be defined** in the **variables top level mapping** in order for an override to work.



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Custom variables can then be used throughout your databricks.yml file. One of the places you can use these is within your targets mapping. This allows you to dynamically modify variable values for each target environment.

<CLICK> For example, let's say we have a custom variable named **target_catalog** that is used to populate a job parameter to read and write data from a specific catalog. You can modify its value for each environment. In this case, when deploying to the development environment, the **target_catalog** variable will use the **catalog_dev** value.

<CLICK> When deploying to production, the **target_catalog** variable will use the **catalog_prod** value.

If you do not specify an override for your variable, it will default to its original value.

<CLICK> Please be aware that a default value for the variable must be defined in the variables top-level mapping for an override to work. If a variable is not assigned a default value, the override will not function as expected.

Benefits of Using Variables

Declarative Automation Bundles



Customizable for Different Environments

Easily modify configurations (e.g., database connections, file paths, etc) for development, staging, and/or production environments.



Reusability Across Databricks Projects

You can use the same asset bundle across multiple teams or workspaces by adjusting only variable values.



Easy Maintenance & Updates

Quickly update assets by modifying variables to ensure consistency and reduce errors.



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

There are several benefits to using variables in your databricks.yml bundle configuration file:

<CLICK> First, you can customizable them for Different Environments: Variables allow you to easily modify configurations, such as catalogs, file paths, and other settings, for development, staging, and production environments.

<CLICK> They provide Reusability Across Databricks Projects: You can use the same asset bundle across multiple teams or workspaces by adjusting only the variable values, promoting consistency and reducing redundancy.

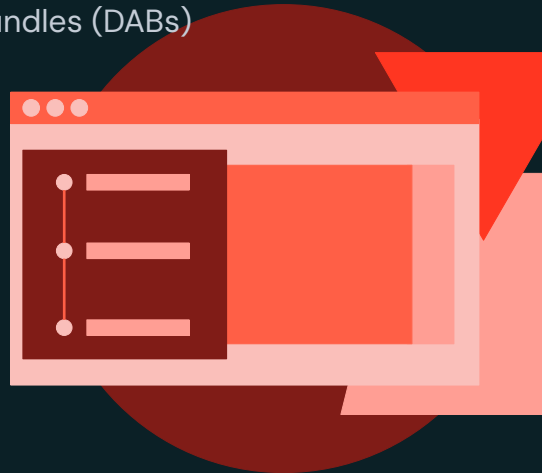
<CLICK> Lastly, you can Easy maintain and update them: With variables, you can quickly update assets and configurations, ensuring consistency across environments and reducing the risk of errors.



Deployment with Declarative Automation Bundles (DABs)

DEMONSTRATION

Deploying a DAB to Multiple Environments



Notebook: /O3 - Deploying a DAB to Multiple Environments/Demo - Deploying a DAB to Multiple Environments



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).



Deployment with Declarative Automation Bundles
(DABs)

LAB EXERCISE

Deploy a DAB to Multiple Environments



Notebook: /O4L – Deploy a DAB to Multiple Environments/Lab – Deploy a DAB to Multiple Environments



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).



Deployment with Declarative Automation Bundles (DABs)

LECTURE

DAB Project Templates Overview



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Declarative Automation Bundles Project Templates

Overview

Default Bundle Template

Use a Databricks **default** bundle template to create your bundle

- Templates:
 - **default-python**
 - default-sql
 - dbt-sql
 - Mlops-stacks

```
databricks bundle init default-python
```



Custom Bundle Template



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Now, let's take a quick look at Declarative Automation Bundles templates.

You have two main options here: you can either use the pre-configured templates provided by Databricks, or you can create your own custom templates to meet your specific needs.

<CLICK> Databricks offers several default bundle templates, including:

- default-python for Python-based projects
- default-sql for SQL-based projects
- dbt-sql for projects involving DBT (Data Build Tool) workflows
- mlops-stacks for machine learning operations stacks

<CLICK> To get started with one of these templates, you can use the Databricks CLI command `databricks bundle init <template_name>`. This command will automatically initialize your asset bundle based on the selected template, saving you time in setting up the configuration from scratch.

One note, when using a bundle template within a Databricks notebooks, you get the entire template. If you are running the CLI command outside of a notebook, a series of prompts enable you to select specific portions of the template.

Declarative Automation Bundles Project Templates

Overview

Default Bundle Template

Use a Databricks default bundle template to create your bundle

- Templates:
 - `default-python`
 - `default-sql`
 - `dbt-sql`
 - `Mlops-stacks`



Custom Bundle Template

- At a minimum, it must have **`databricks_template_schema.json`** and **`databricks.yml.tpl`**
- You can add user prompts, specific folder structure and more
- To use a custom bundle template, pass its local path or remote URL to the Databricks CLI bundle init command.

```
databricks bundle init  
/projects/templates/test-template
```



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

In addition to Databricks' default templates, you can create custom bundle templates specific to your organization.

Although we won't go into the details of creating custom templates in this course, here are a few key points:

- Custom templates require at least `databricks_template_schema.json` and `databricks.yml.tpl`.
- You can include user prompts, define folder structures, and customize settings.

<CLICK> To use a custom template, simply pass the local path or remote URL of the template to the Databricks CLI bundle init command.

Custom templates help organizations create and manage bundles in a consistent and repeatable way, establishing folder structures, tasks, and DevOps infrastructure-as-code (IaC) for development and deployment.



Deployment with Declarative Automation Bundles
(DABs)

LAB EXERCISE

Use a Databricks Default DAB Template



Notebook: /O5L – Use a Databricks Default DAB Template/Lab – Use a Databricks Default DAB Template



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).



Deployment with Declarative Automation Bundles (DABs)

LECTURE

CI/CD Project Overview with DABs



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

In this lecture, we're going to walk through how to implement a CI/CD workflow using **Declarative Automation Bundles (DABs)**. We'll cover how to plan a project, set up testing strategies, and deploy workloads efficiently across different environments.

Planning the Project

Requirements

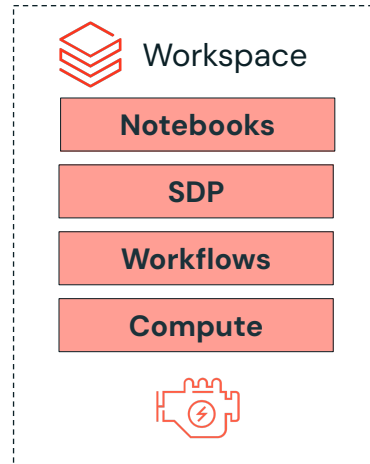
Deliverable

- Visualize Health Data

Tasks

- Ingest daily incremental CSV files to a bronze table
- Create a clean silver table
- Create gold tables to share with consumers

Databricks Assets



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

To start, let's discuss what **CI/CD** is and how **Declarative Automation Bundles (DABs)** streamline the process.

- **Continuous Integration (CI):** Automating code integration and testing.
- **Continuous Delivery/Deployment (CD):** Ensuring code is always in a deployable state.

Declarative Automation Bundles simplify this process by allowing us to **define, version, and deploy assets in a structured way using the Databricks CLI**.

When planning a project, it's crucial to define:

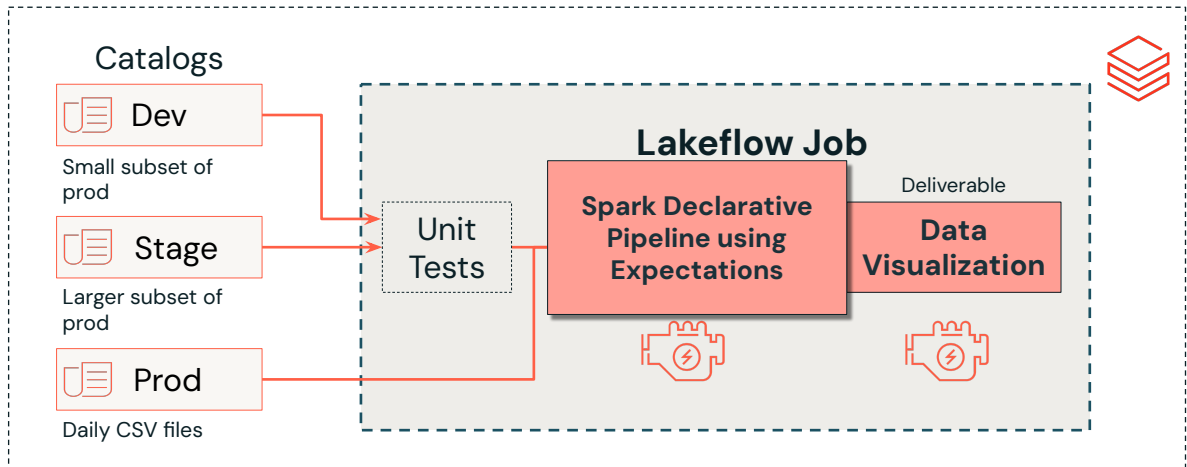
- **Deliverables** → What are we building?
- **Tasks** → What are the key steps to achieve this?
- **Databricks Assets** → What components are required?

In this case, we are **visualizing health data**. The key tasks include:

1. **Ingesting a daily incremental CSV file** into a **bronze table**.
2. **Cleaning the data** and moving it to a **silver table**.

Planning the Project

Course Project Architecture



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Now that we have an understanding of our project, our data, and how to isolate environments, let's dive into the project setup.

In this project we will create a catalog for each environment: dev, stage, and prod.

The dev catalog will contain a small, static subset of our production data, used for development and initial testing.

The stage catalog will hold a larger subset of production data, enabling more comprehensive testing as we move through our CI/CD process.

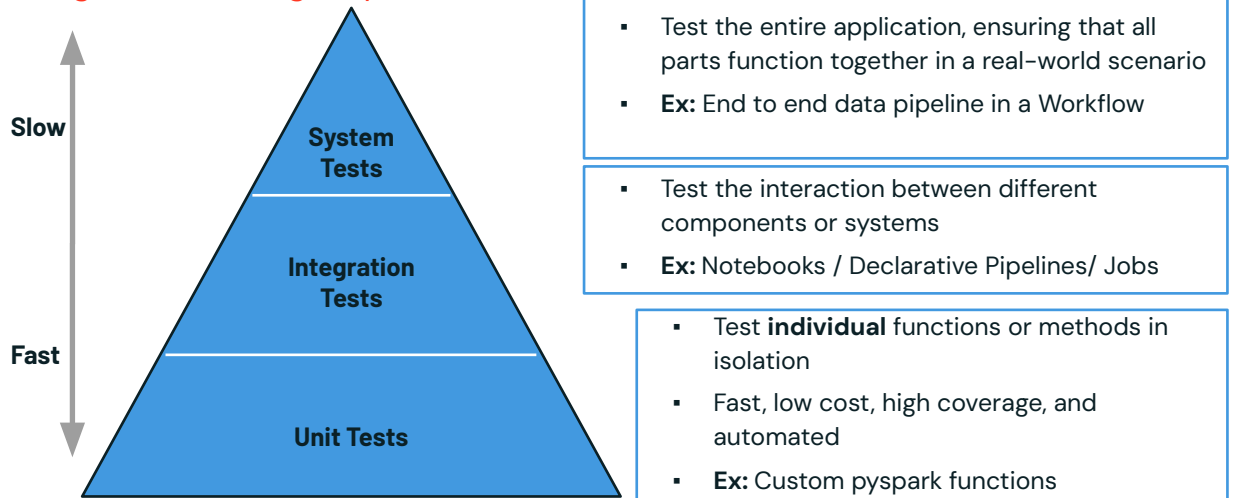
Finally, the prod catalog will contain the live production data that we rely on for final operations.

Our workflow starts by developing on the dev data, running unit and integration tests as we progress through the pipeline. The pipeline is set up with Databricks Workflows, which execute the unit tests, Lakeflow Declarative pipeline, and final visualizations.

As we test the pipeline through each stage, we will ensure everything is functioning correctly before deploying to production.

Review the Role of CI/CD Testing

High-level Testing Steps



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Within CI/CD there are testing steps you should following within what's called the testing pyramid. The testing pyramid categorises different tests, unit tests, integration tests and system tests. Testing your code is extremely important.

<CLICK> The base of the pyramid are unit tests which test individual functions or methods in isolation. Since they small individual functions, they typically can run quickly, frequently and automatically, ensuring that the functions work as expected. Unit tests form the foundation because they are inexpensive and provide the broadest coverage. For example, testing if a pyspark method works as expected.

<CLICK> Next is integration tests test the interaction between different components or systems. These are typically slower and more costly than unit tests, but provide greater assurance that components work together correctly. Within Databricks these typically will revolve around using Notebooks, Lakeflow Declarative Pipelines and or Lakeflow Jobs. For example, testing whether a pyspark method and pipeline work correctly.

<CLICK> Lastly system tests test the entire application, ensuring that all parts function together in a real-world scenario. These are typically slow, costly, and often run in a production-like environment. For example, for our end to end data pipeline, testing whether the data data pipeline works as expected within a Workflow, creating our desired results.

Review of Unit Testing with pytest

Pytest -is popular testing framework for Python that makes it easy to write simple and scalable test cases.

Uses Simple Syntax

Minimal syntax, just define functions starting with **test_**

Provides Assertions

Use **assert** statements to provide detailed error messages on failure

Automatic Discovery

Finds and runs all tests **automatically** with a simple configuration

Rich Ecosystem

Extend functionality with plugins for coverage, parallel tests, and more

This course provides a simple introduction to **pytest**. There are many testing frameworks available, select the one that best meets your organization's needs.



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Next like talk about the pytest unit testing framework.

Pytest -is popular testing framework for Python that makes it easy to write simple and scalable test cases. It provides a variety of benefits for executing your unit tests.

First up, pytest has a very simple syntax. You don't need to worry about complex setup. Just write test functions that start with `test_`, and pytest will automatically pick them up and run them.

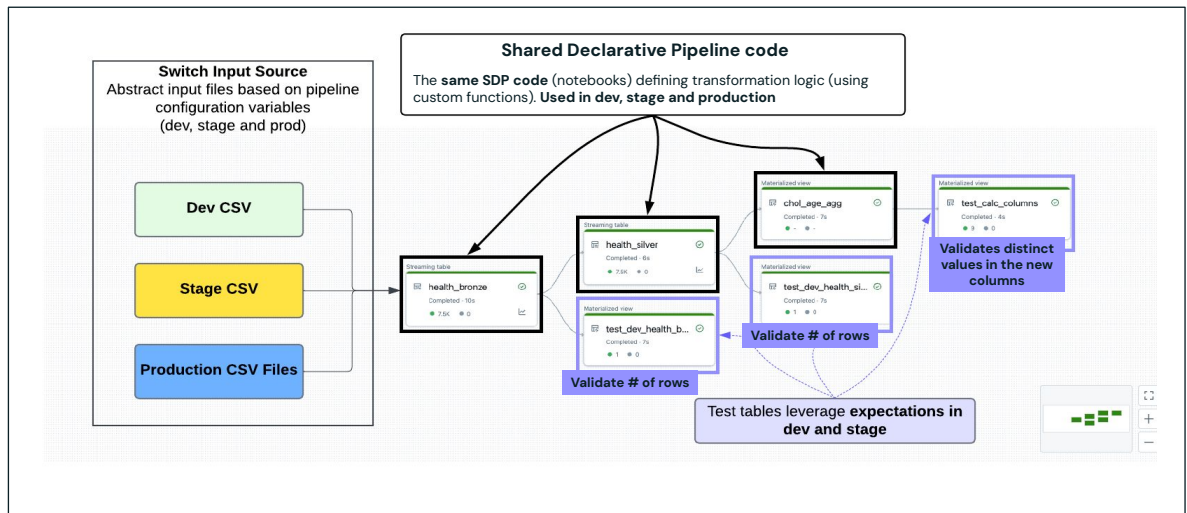
Next, pytest leverages Python's built-in assert statements (or you can use other assert statements like in pyspark). These are simple to use but provide detailed error messages when a test fails, making your function debugging much easier and faster by understanding what went wrong.

Pytest also automatically discovers and runs all your tests. There's no need to manually configure which tests to run. Just name your test files and functions with the `test_` prefix, and pytest will take care of the rest.

Lastly, pytest has a rich ecosystem of plugins to extend its functionality. Whether you need test coverage reports, parallel test execution, or integration with other tools, there's a plugin for almost anything, allowing you to customize pytest to fit your needs.

This course provides a simple introduction to pytest. There are many testing frameworks available, select the one that best meets your organization's needs.

Review of Pipeline Expectations for Integration Tests



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://apache.org/).

Summary of Pipeline Expectations:

[MISSING]

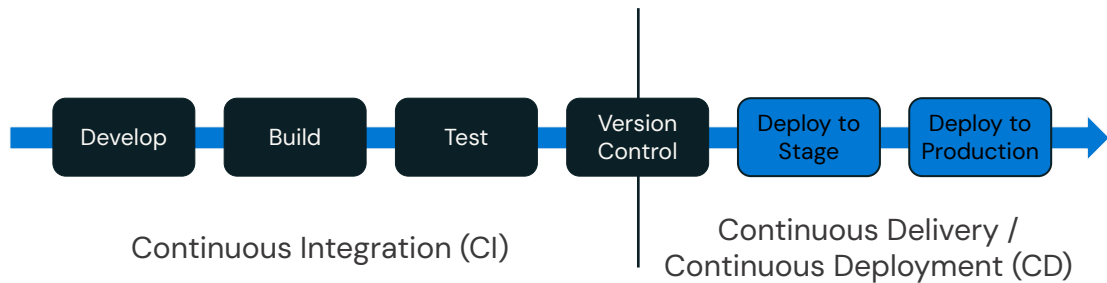
Provide links:

- Look in here for links

<https://dbc-d9be2316-40bd.cloud.databricks.com/editor/notebooks/634209448007576?o=3434287344177969#command/634209448007577>

The Role of CI/CD in DevOps

High-Level CI/CD Workflow Overview



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

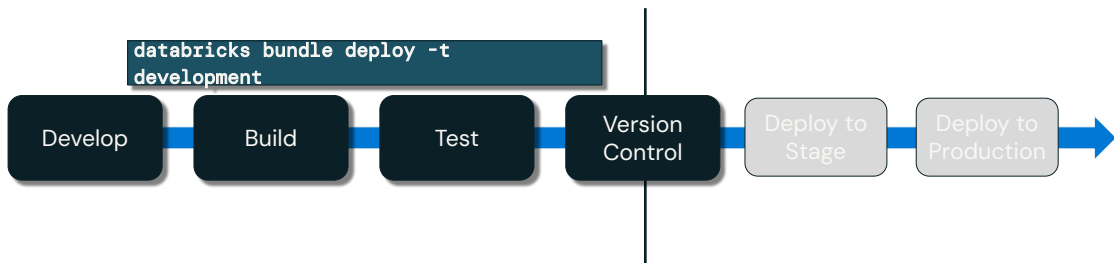
Looking at the high level overview of CI/CD we want to combine continuous integration (develop, build, test and version control) with continuous delivery/deployment (deploying the project to stage and production). Automating this entire process to develop quickly and error free.

In the end, the CI/CD process streamlines development of your data pipelines by automating testing and deployment, leading to faster, more reliable releases.

This approach minimizes manual errors, improves collaboration, and ensures high-quality software delivered quickly and consistently.

CI/CD with DABs

High-Level CI/CD Workflow Overview



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

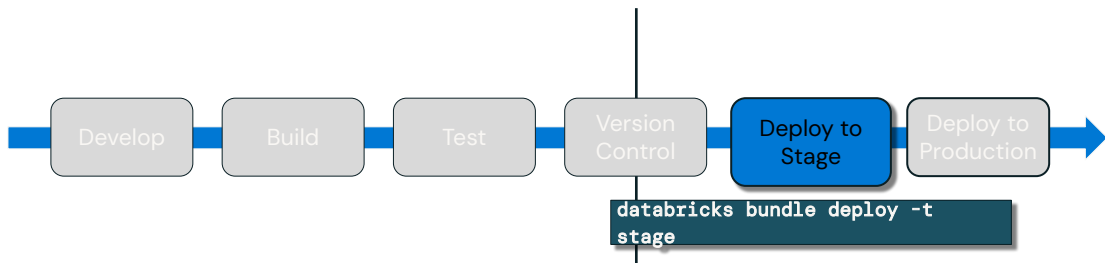
Looking at the high level overview of CI/CD we want to combine continuous integration (develop, build, test and version control) with continuous delivery/deployment (deploying the project to stage and production). Automating this entire process to develop quickly and error free.

In the end, the CI/CD process streamlines development of your data pipelines by automating testing and deployment, leading to faster, more reliable releases.

This approach minimizes manual errors, improves collaboration, and ensures high-quality software delivered quickly and consistently.

CI/CD with DABs

High-Level CI/CD Workflow Overview



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

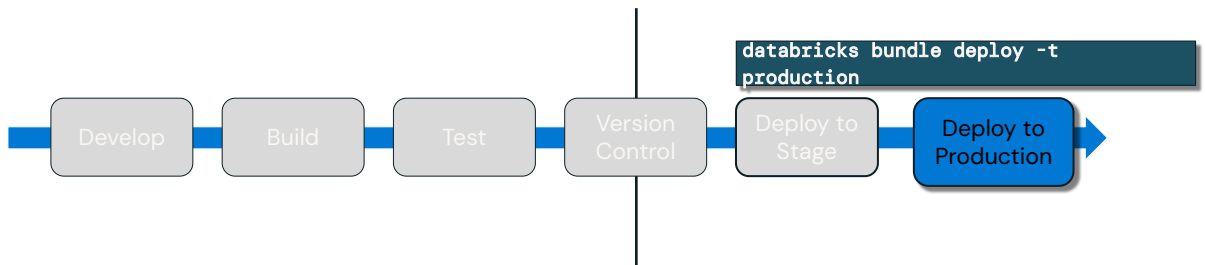
Looking at the high level overview of CI/CD we want to combine continuous integration (develop, build, test and version control) with continuous delivery/deployment (deploying the project to stage and production). Automating this entire process to develop quickly and error free.

In the end, the CI/CD process streamlines development of your data pipelines by automating testing and deployment, leading to faster, more reliable releases.

This approach minimizes manual errors, improves collaboration, and ensures high-quality software delivered quickly and consistently.

CI/CD with DABs

High-Level CI/CD Workflow Overview



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Looking at the high level overview of CI/CD we want to combine continuous integration (develop, build, test and version control) with continuous delivery/deployment (deploying the project to stage and production). Automating this entire process to develop quickly and error free.

In the end, the CI/CD process streamlines development of your data pipelines by automating testing and deployment, leading to faster, more reliable releases.

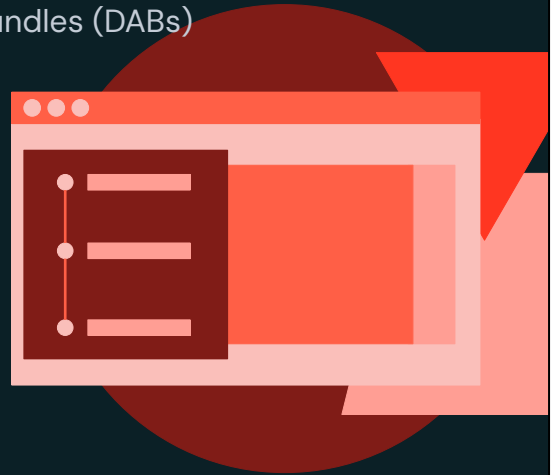
This approach minimizes manual errors, improves collaboration, and ensures high-quality software delivered quickly and consistently.



Deployment with Declarative Automation Bundles (DABs)

DEMONSTRATION

Continuous Integration and Continuous Deployment with DABs



Notebook: /06- Continuous Integration and Continuous Deployment with DABs/Demo - CI/CD with DABs



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Next, we will demonstrate how to use CI/CD with DABs. Before beginning the demonstration, let's first investigate the folder structure you'll be working with.

Full Project Outline

Folder Architecture

Full Project/

1 Create a project folder.

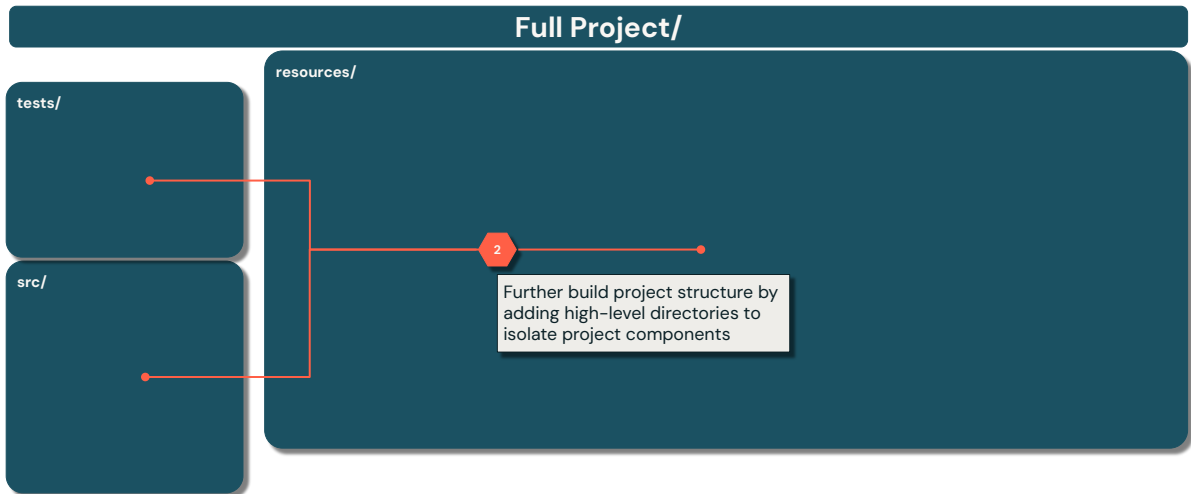


© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

The first step in setting up a Declarative Automation Bundle is to **create a structured project directory**.

Full Project Outline

Folder Architecture



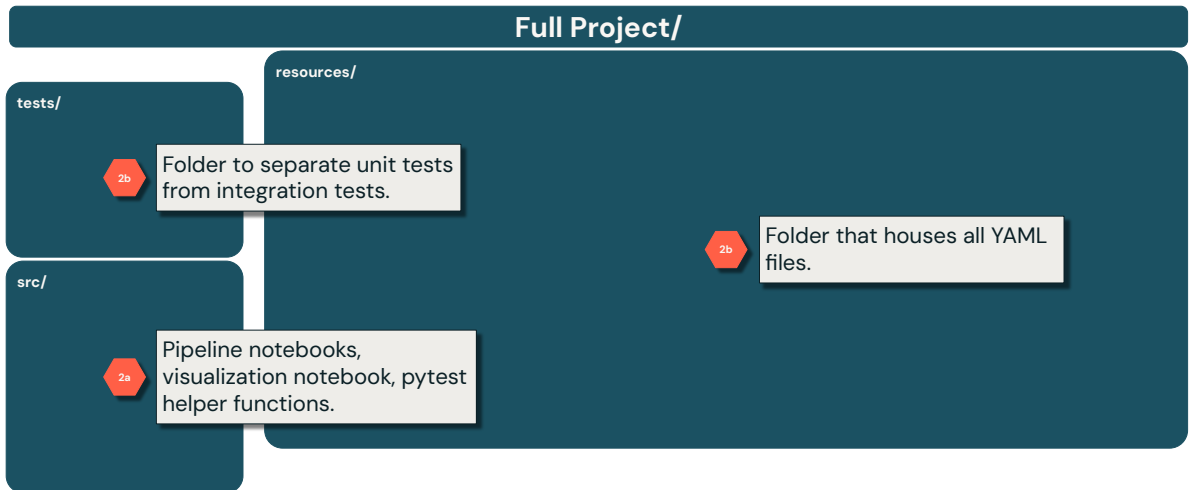
© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

At the **root level**, the project follows this layout:

```
Full Project/  
├── resources/  
├── src/  
├── tests/  
└── databricks.yml
```

Full Project Outline

Folder Architecture



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

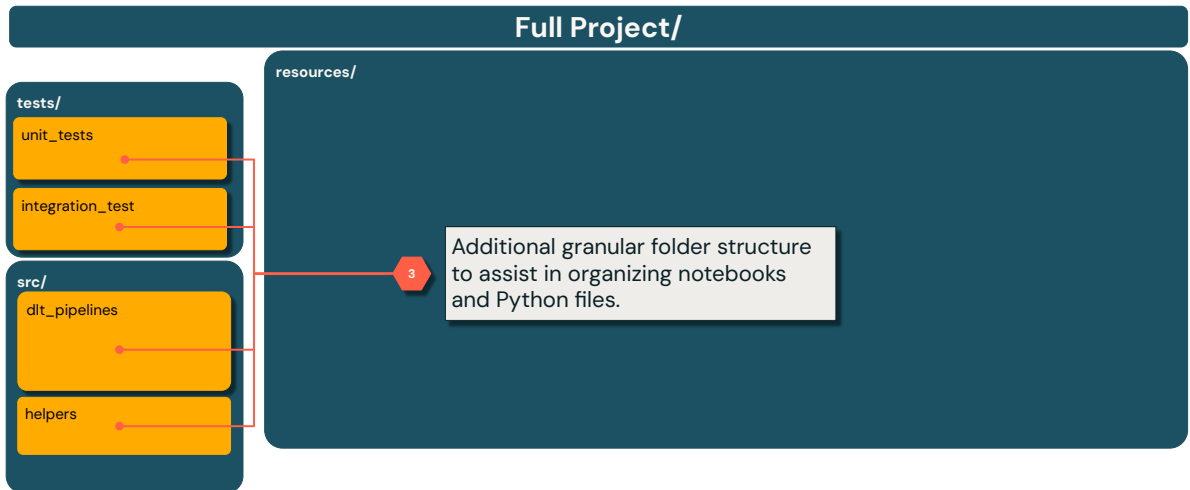
Each directory serves a specific purpose:

- **resources/** – Stores YAML configuration files for workflows and pipelines.
- **src/** – Contains all **source code**, such as Python and SQL notebooks.
- **tests/** – Includes **unit tests and integration tests** for validation.
- **databricks.yml** – The required **configuration file** that defines the Declarative Automation Bundles.

This structure keeps code, configurations, and tests **organized** and **easy to manage**.

Full Project Outline

Folder Architecture



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Testing is a crucial part of CI/CD, ensuring that **code works before deployment**.

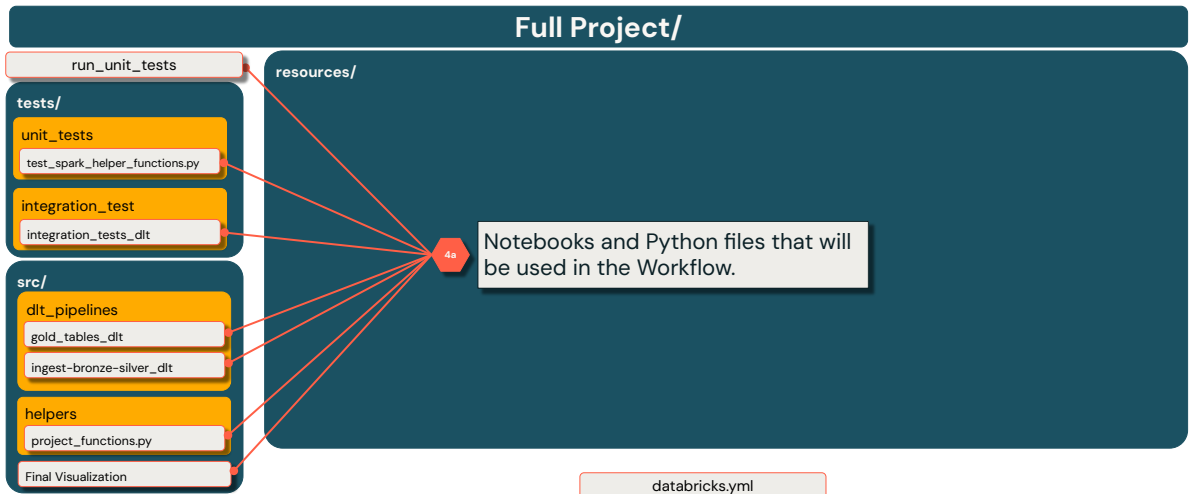
- **Unit tests** validate individual components like functions or transformations.
- **Integration tests** ensure that different **Databricks assets (jobs, workflows, pipelines)** work together as expected.

All tests are run **automatically in the CI pipeline** to catch errors early.

The **src/** directory holds the actual **implementation logic** for the project.

Full Project Outline

Folder Architecture

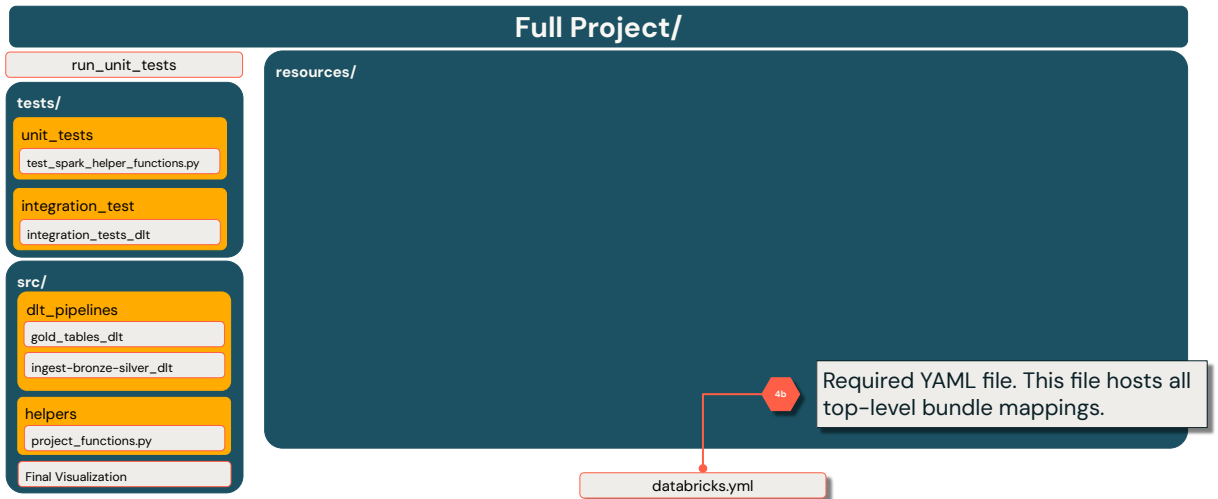


© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Once we have our folder structure for **src/** and **tests/**, we build out the notebooks and python files that will be used with this project.

Full Project Outline

Folder Architecture



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Next, we will add the **databricks.yml** file that hosts all top-level bundle mappings for our project.

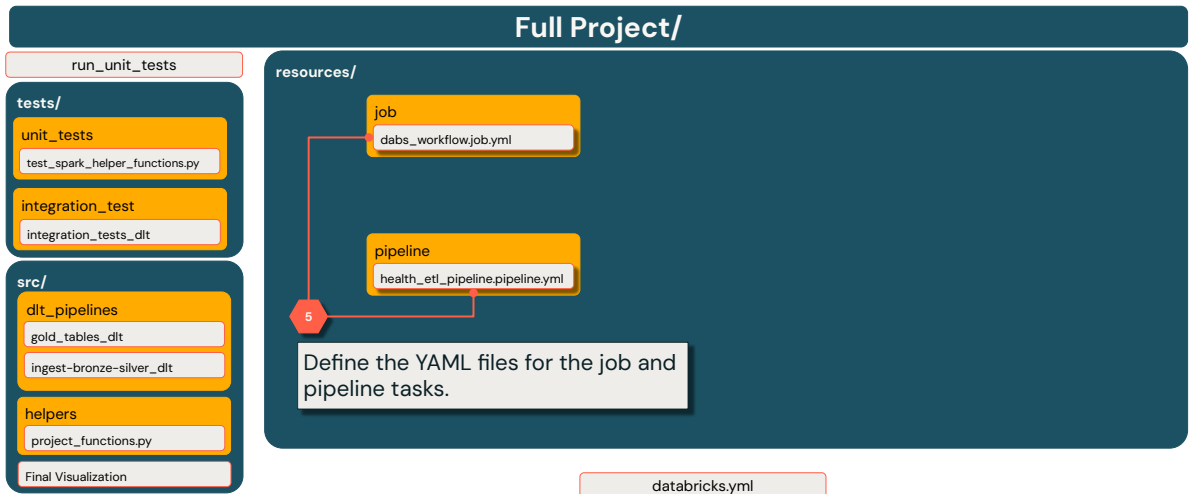
The **databricks.yml** file is the **core configuration** of the Declarative Automation Bundle.

It includes:

- **Bundle name**
- **Resources** (workflows, pipelines, models)
- **Targets** (development, staging, production)
- **Variables** (paths, authentication details)

Full Project Outline

Folder Architecture

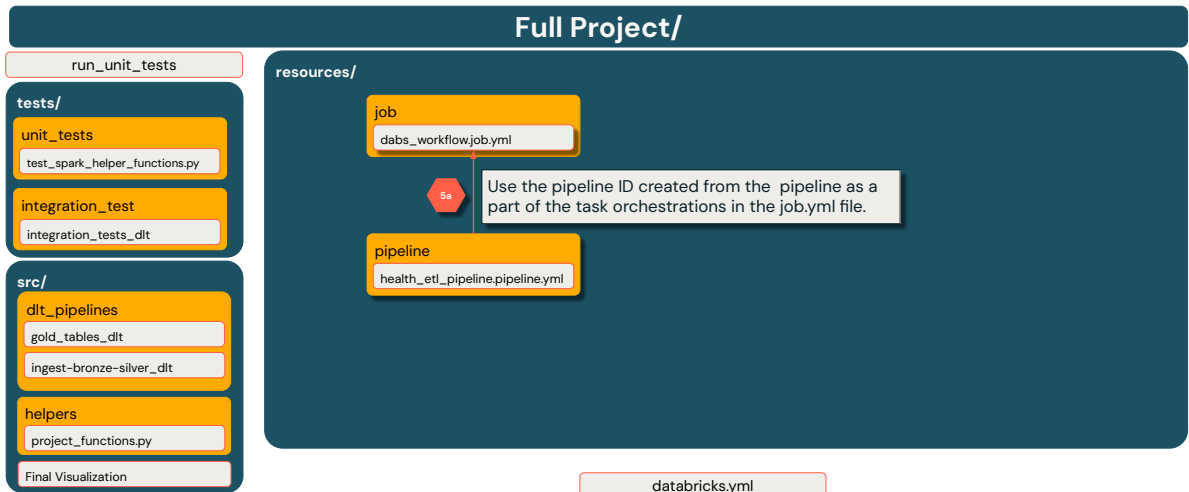


© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Next, let's take a look at the resources folder. The **resources/** directory is where all **YAML configuration files** for Declarative Automation Bundles are stored.

Full Project Outline

Folder Architecture



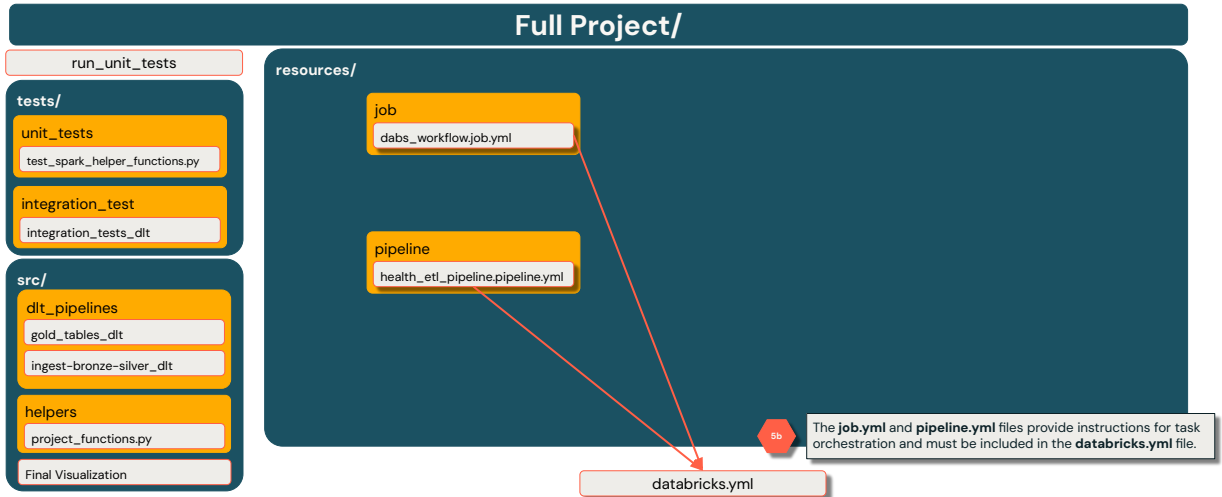
© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

dabs_workflow.job.yml – Defines **job tasks**, execution order, and dependencies.

health_etl_pipeline.pipeline.yml – Defines **DLT pipelines**, specifying transformations and output tables.

Full Project Outline

Folder Architecture

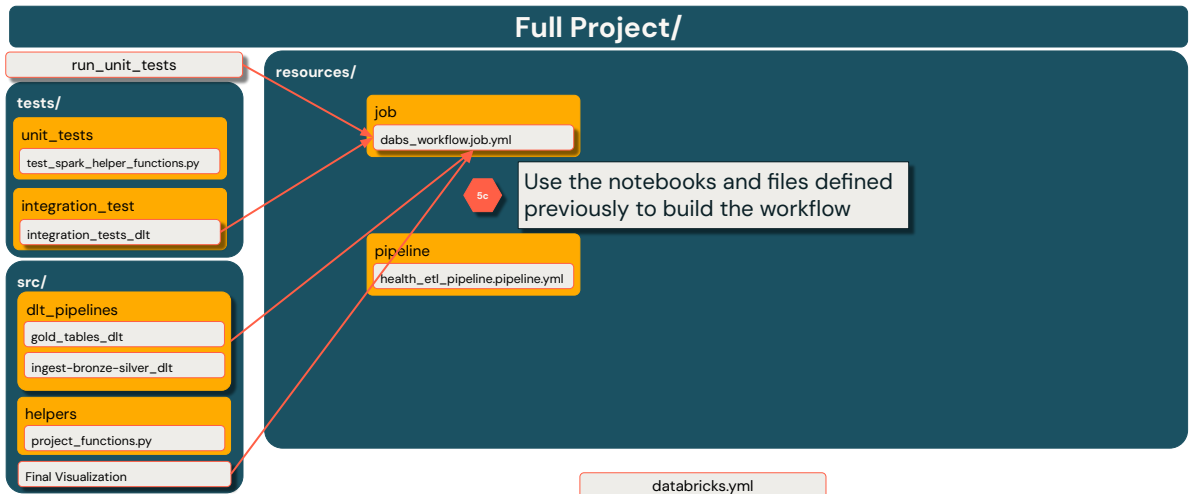


© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

We include the job and pipeline yaml files in the databricks.yml file.

Full Project Outline

Folder Architecture

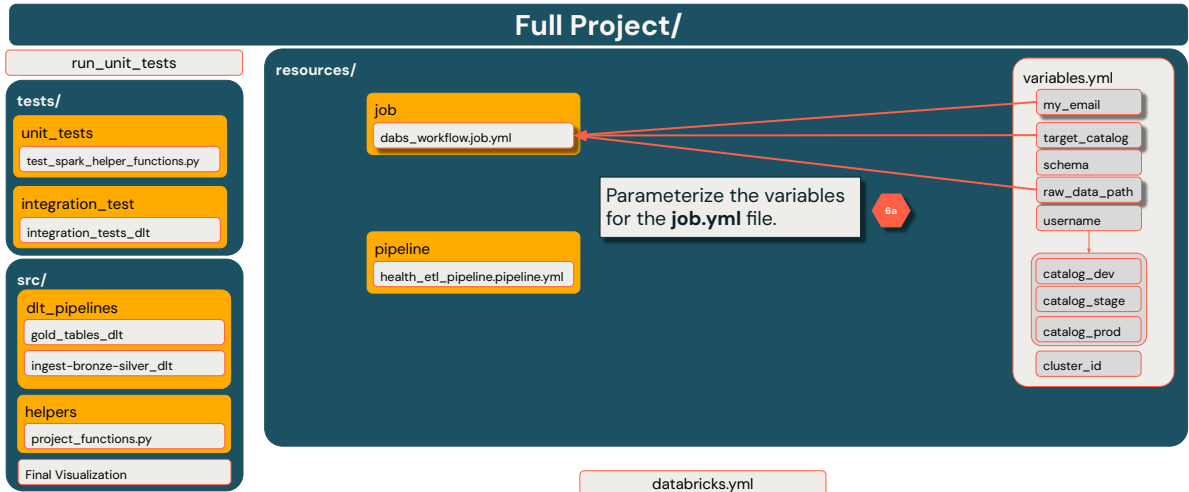


© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

We see here all the files and folders that are to be used inside the job.yml file - this brings together the complete workflow that will be deployed and ran within Databricks.

Full Project Outline

Folder Architecture

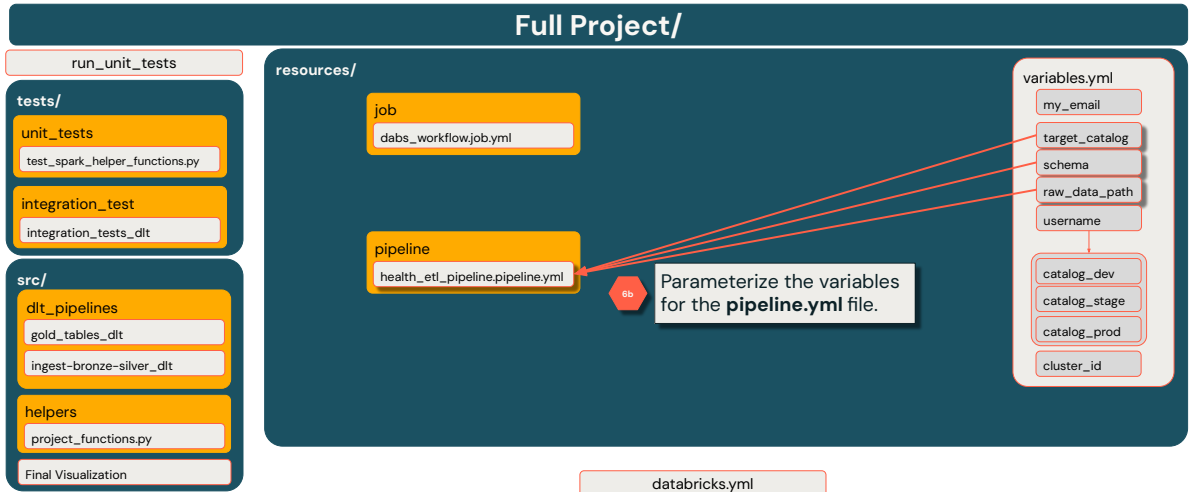


© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

To help modularize the setup for the demo, we have separated the variables from the databricks.yml file. When you edit the configurations for the bundle, you will do so by manipulating the databricks.yml file or the variables.yml file - each have their own purpose.

Full Project Outline

Folder Architecture

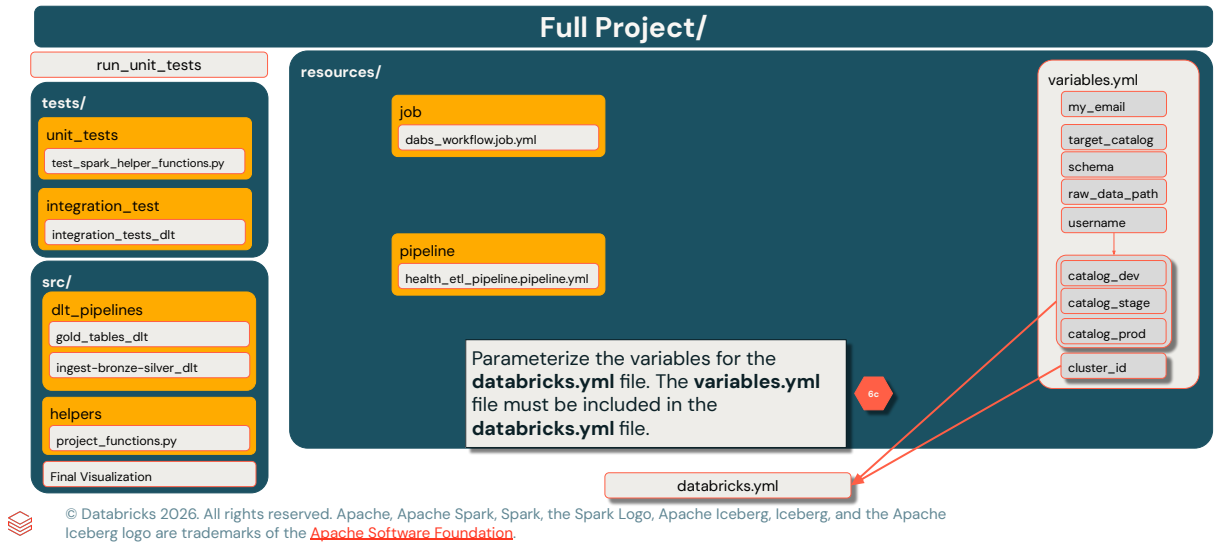


© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Similarly, we configure any variable that is to be used in the `pipeline.yml` file as well.

Full Project Outline

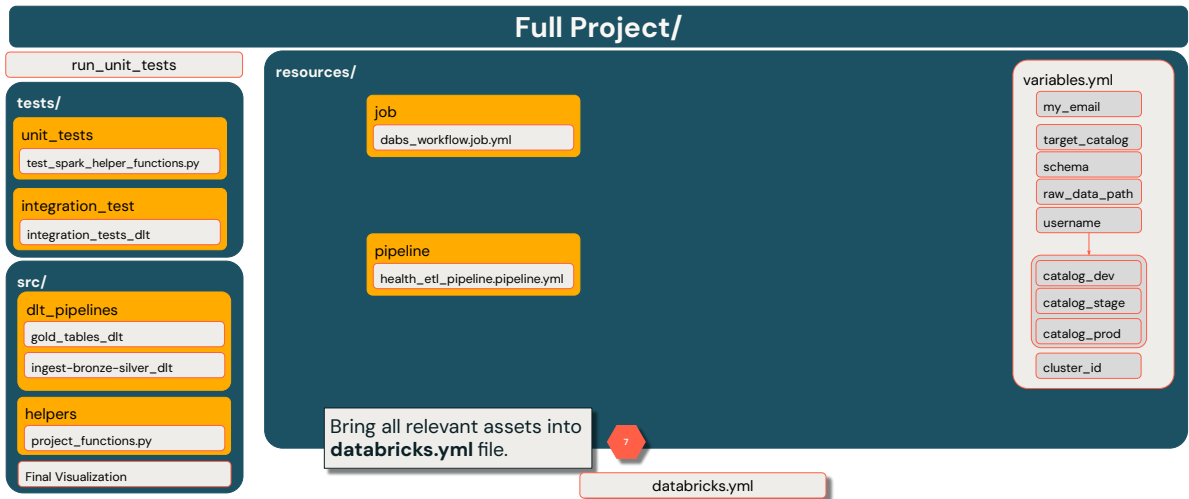
Folder Architecture



The `databricks.yml` file is the core configuration YAML file for high-level orchestration. The `variables.yml` file is used to fine-tune the bundle configuration like defining the default user and compute cluster.

Full Project Outline

Folder Architecture



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

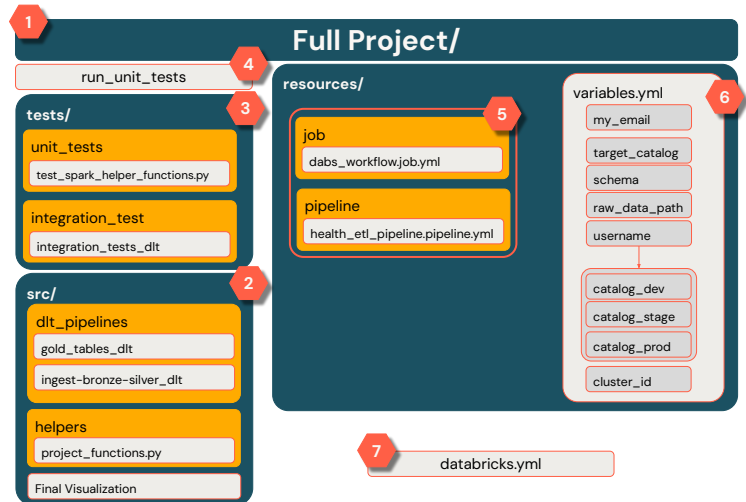
Note, that we can certainly overwrite the default variable values within the `databricks.yml` file. Once all bundle assets have been assembled, we are ready to validate, deploy, and run the bundle using the Databricks CLI.

Full Project Outline

Summary

CI/CD with DABs Summary

- 1 Define root directory for the project
- 2 Incorporate tested notebooks – these notebooks should be tested in isolation prior to migrating to DABs
- 3 Set up a folder for isolating different tests, e.g. unit tests and integration tests
- 4 Define any other notebooks
- 5 Define YAML files for jobs and pipeline tasks
- 6 Parameterize all YAML files with variables.yml if possible
- 7 Define databricks.yml file



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

To summarize, this demonstration will cover

- ✓ **The folder structure of a Declarative Automation Bundle**
- ✓ **How source code, tests, and configurations are organized**
- ✓ **How to validate, deploy, and run a bundle using the CLI**

By following this structure, teams can **efficiently manage, test, and deploy Databricks projects** in a reliable CI/CD pipeline.



Deployment with Declarative Automation Bundles (DABs)

LAB EXERCISE

Adding ML to Engineering Workflows with DABs

This is an optional bonus lab for those interested in deploying a machine learning experience using DABs. Feel free to explore it on your own after class.



Notebook: /O7L Bonus – Adding ML to Engineering Workflows with DABs/Lab – Make a Machine Learning Task



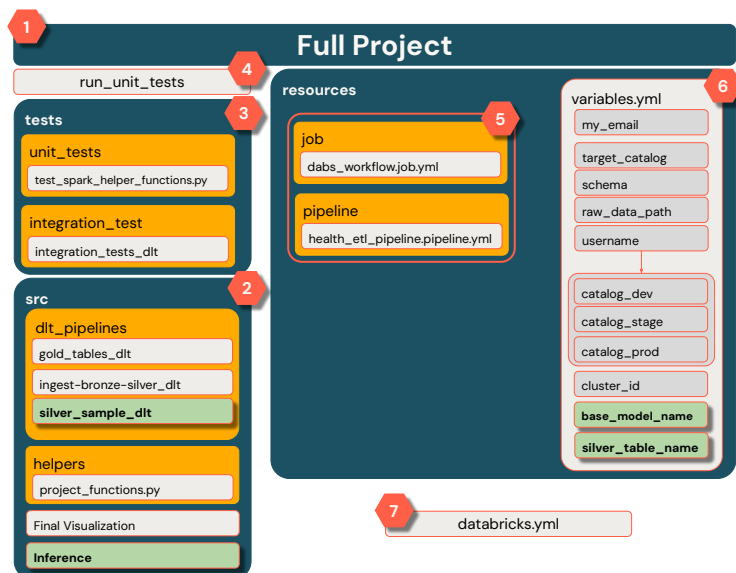
© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Lab Outline

Incorporate an ML Task

CI/CD with DABs Summary

- 1 Define root directory for the project
- 2 Incorporate tested notebooks – these notebooks should be tested in isolation prior to migrating to DABs
- 3 Set up a folder for isolating different tests, e.g. unit tests and integration tests
- 4 Define any other notebooks
- 5 Define YAML files for jobs and pipeline tasks
- 6 Parameterize all YAML files with variables.yml if possible
- 7 Define databricks.yml file

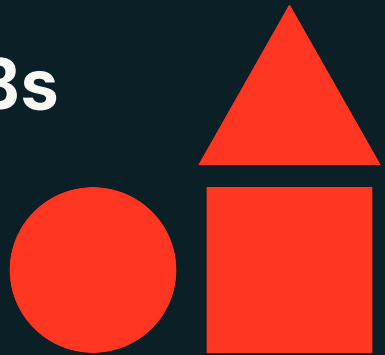


© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).



Doing More with DABs

Automated Deployment with Declarative Automation Bundles



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Agenda

Doing More with DABs

- **Developing Locally with Visual Studio Code (VSCode)**
- **CI/CD Best Practices for Data Engineering**
- **Next Steps: Automated Deployment with GitHub Actions**



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Copy over this table if necessary.

Section Learning Objectives

- Understand core components of developing locally with Visual Studio Code (VS Code)
- Understand how to use Databricks Connect with VS Code
- Discuss the Databricks Extension within VS Code
- Understand core principles of best practices for CI/CD
- Discuss common development patterns with GitHub Actions and common 3rd party tools



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).



Doing More with DABs

LECTURE

Developing Locally with Visual Studio Code (VSCode)



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Developing Locally with (VSCode)

Developer Tools Overview



Databricks CLI

- Ideal for shell scripting & lightweight command-line tasks
- Useful in development and CI/CD processes
- Supports unified authentication (OAuth, PATs, etc.)
- More interactive debugging compared to Notebooks



Databricks Connect V2



Databricks VSCode Extension



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

For many developers, using an IDE like VSCode is the preferred choice. With VSCode, you can interact with your Databricks environment in multiple ways, allowing you to stay within the IDE and avoid switching to the Databricks browser UI.

Let's quickly review some of the tools you can use within VSCode.

<CLICK> First, you can install the Databricks CLI locally.

The Databricks CLI allows for quick interactions with Databricks through the command line. It's ideal for tasks such as:

- Shell scripting and lightweight command-line tasks
- Development and CI/CD processes
- Supporting unified authentication (OAuth, PATs, etc.)
- Providing more interactive debugging compared to using the CLI in Notebooks

Developing Locally with (VSCode)

Developer Tools Overview



Databricks CLI

- Ideal for shell scripting & lightweight command-line tasks
- Useful in development and CI/CD processes
- Supports unified authentication (OAuth, PATs, etc.)
- Limited interactive debugging compared to IDEs



Databricks Connect V2

- Runs Apache Spark code remotely on a Databricks cluster from a local environment
- Ideal for interactive debugging and remote Spark job execution
- Easy setup with pip install databricks-connect>= your version



Databricks VSCode Extension



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Next, you can install and use Databricks Connect v2.

Databricks Connect v2 allows you to interact with Databricks from anywhere, enhancing flexibility and productivity.

With Databricks Connect v2, you can:

- Run Apache Spark code remotely on a Databricks cluster from a local environment
- Ideal for interactive debugging and remote Spark job execution
- Easy setup with pip install databricks-connect>=<your version>

Developing Locally with (VSCode)

Developer Tools Overview



Databricks CLI

- Ideal for shell scripting & lightweight command-line tasks
- Useful in development and CI/CD processes
- Supports unified authentication (OAuth, PATs, etc.)
- Limited interactive debugging compared to IDEs



Databricks Connect V2

- Runs Apache Spark code remotely on a Databricks cluster from a local environment
- Ideal for interactive debugging and remote Spark job execution
- Easy setup with pip install databricks-connect>=13.0.28



Databricks VSCode Extension

- Integrates Databricks into VS Code for batch and interactive development
- Simplifies setup with resource explorers and Databricks CLI
- Supports running and debugging Python files on Databricks clusters
- Keeps developers within VS Code, and provides DAB features



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Lastly, you can use the Databricks VSCode Extension.

This extension enhances your IDE experience by integrating Databricks functionalities directly into Visual Studio Code.

With the VSCode extension, you can:

- Integrate Databricks into VS Code for both batch and interactive development
- Simplify setup with resource explorers and the Databricks CLI
- Run and debug Python files directly on Databricks clusters
- Stay within VS Code while accessing additional bundle user interface features

Databricks Extension for (VSCode)

Overview



Simple Setup

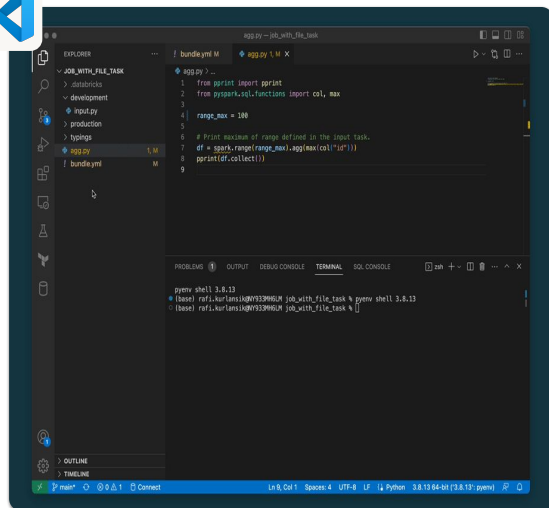
Find us on the VS Code Marketplace and get connected to compute in minutes

Native Experience

Write code using the productivity features you love from VS Code

Run on Databricks

Execute batch workloads or start interactively debugging directly from your IDE



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

The VSCode extension has a simple setup from the VSCode marketplace, allowing you to get connected to compute in minutes.

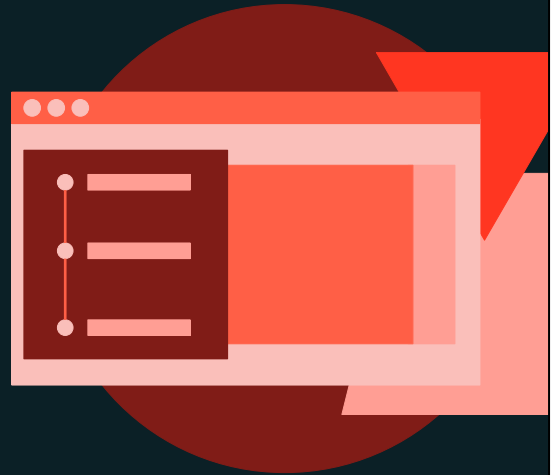
It provides a native experience for developers who prefer using VSCode and enables you to run code on Databricks directly from your IDE.



Module Name

DEMONSTRATION

Using VSCode with Databricks



Notebook: /O8 - Using VSCode with Databricks/O8 - Using VSCode with Databricks



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).



Doing More with DABs

LECTURE

CI/CD Best Practices for Data Engineering



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

In this section, we will cover CI/CD best practices for Data Engineering

CI/CD Best Practices for Data Engineering

Summary



Define a Clear Testing Strategy

Establish and automate unit, integration and end to end tests



Automate Deployment with DABs

Use DABs for consistent, repeatable, and automated deployments in Databricks



Implement a Version Control Strategy

Use a version control system like git to manage code and define branching strategies



Monitor and Optimize CI/CD Pipelines

Continuously monitor the performance of your CI/CD pipelines



Establish a Code Review Process

Set up mandatory code reviews to ensure the quality of changes before they are merged



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

When it comes to implementing CI/CD strategies in an organization, especially in data engineering, there are a few key best practices to think about.

First, define a clear testing strategy that includes unit, integration, and end-to-end tests. Automating these tests ensures your code functions correctly and that data integrity is maintained before deployment. This reduces the risk of errors and improves the overall quality of your codebase.

<CLICK> Next, implement a solid version control strategy using systems like GitHub to manage your code, notebooks, and configuration files. A good version control strategy is essential for collaboration across teams. I recommend defining a branching strategy to help manage different stages of development, making it easier to track changes and resolve conflicts.

<CLICK> It's also crucial to establish a robust code review process. Peer reviews help ensure that changes are thoroughly vetted before they are merged. Additionally, CI tools can automatically trigger builds on pull requests, offering a quick way to validate changes and catch issues early on.

<CLICK> Next, leverage Declarative Automation Bundles (DABs) for consistent and repeatable deployments. Automating the deployment process minimizes human error and speeds up the deployment cycle. DABs ensure that the right code is always deployed to the correct environment, maintaining consistency across your projects.

<CLICK> Finally, continuously monitor the performance of your CI/CD pipelines. Optimizing for speed, reliability, and scalability ensures your pipeline runs as efficiently as possible. Gather feedback from developers, identify bottlenecks, and work toward improving automation efficiency.

By implementing these practices, you'll create a streamlined, reliable, and scalable CI/CD process within your Databricks environment. This will not only improve your development cycles but also increase the overall quality of your data-driven products.

CI/CD Best Practices for Data Engineering

Summary



Implementing CI/CD

Requires alignment across teams and establishment of robust processes that ensure continuous improvement and quality

For more information check out the [Best practices for operational excellence](#)



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

In conclusion, implementing CI/CD practices requires alignment across teams and the establishment of set processes that ensure continuous improvement and quality.

For more detailed best practices, I recommend checking out the Databricks Documentation: Best Practices for Operational Excellence.



Doing More with DABs

LECTURE

Next Steps: Automated Deployment with GitHub Actions



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Deployment Patterns with DABs and GitHub Actions



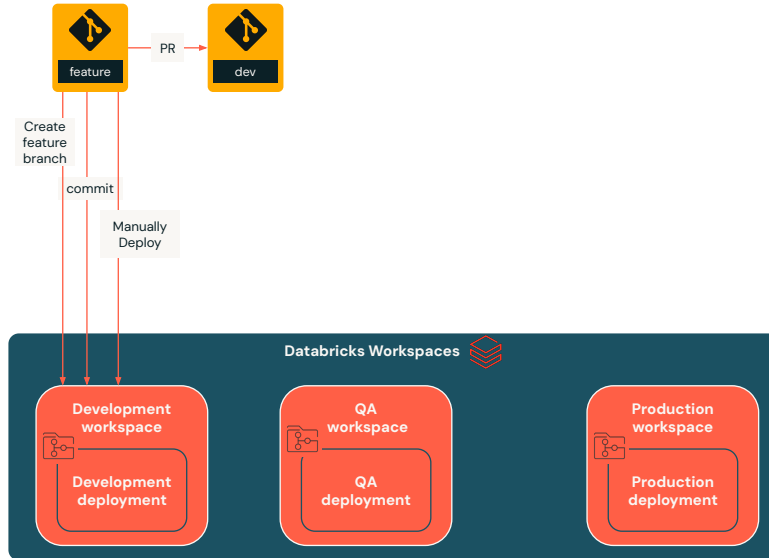
Introduction:

- Let's walk through a high-level overview of how we integrate GitFlow (which is an alternative Git branching model that involves the use of feature branches and multiple primary branches) with Declarative Automation Bundles, or DABs. This workflow streamlines the process of developing, testing, and deploying Databricks projects by leveraging GitFlow's structured branching model and GitHub Actions for automation.

Creation of Feature Branches:

- The process begins on the very left hand side with the developer creating a feature branch from the `dev` branch. This is where individual features or bug fixes are developed in isolation. Developers can deploy code directly to the DEV Databricks workspace using the `databricks-cli`

Deployment Patterns with DABs and GitHub Actions

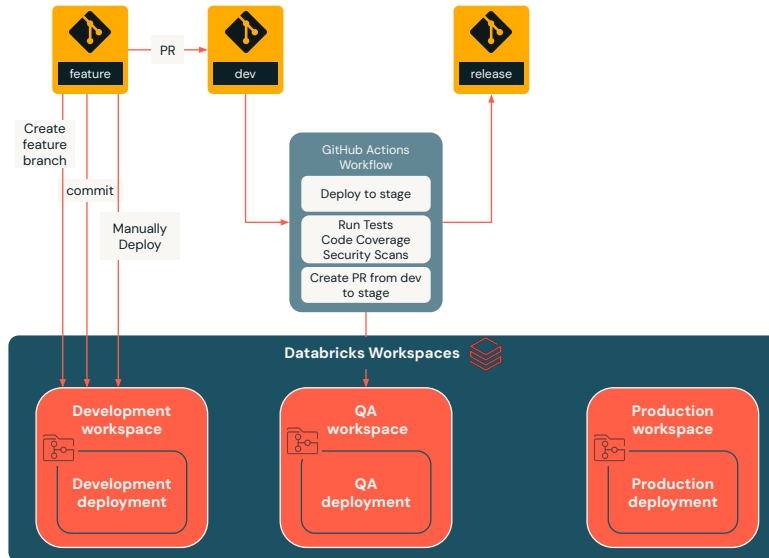


© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Pull Request Workflow:

- Once the development on a feature branch is complete, a pull request is opened. After the code is peer-reviewed, it is merged into the develop branch.

Deployment Patterns with DABs and GitHub Actions



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

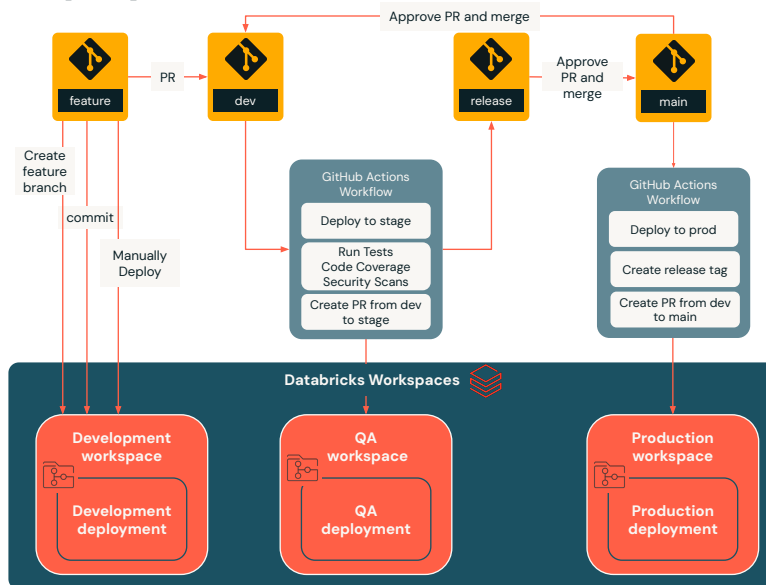
Deployment to QA and Production:

- After merging into develop, the next step involves triggering the GH Actions workflow for Drafting a release. This deploys the changes to the QA environment using DABs ,runs Tests, code coverage checks and security scans and finally creates the versioned release branch. It also creates a PR for the release branch to be merged into main.

<click>

- After the Release PR is closed, it triggers another GitHub actions workflow that deploys the bundle to PROD, creates a Release Tag and eventually, a PR for main branch to be merged into DEV.

Deployment Patterns with DABs and GitHub Actions

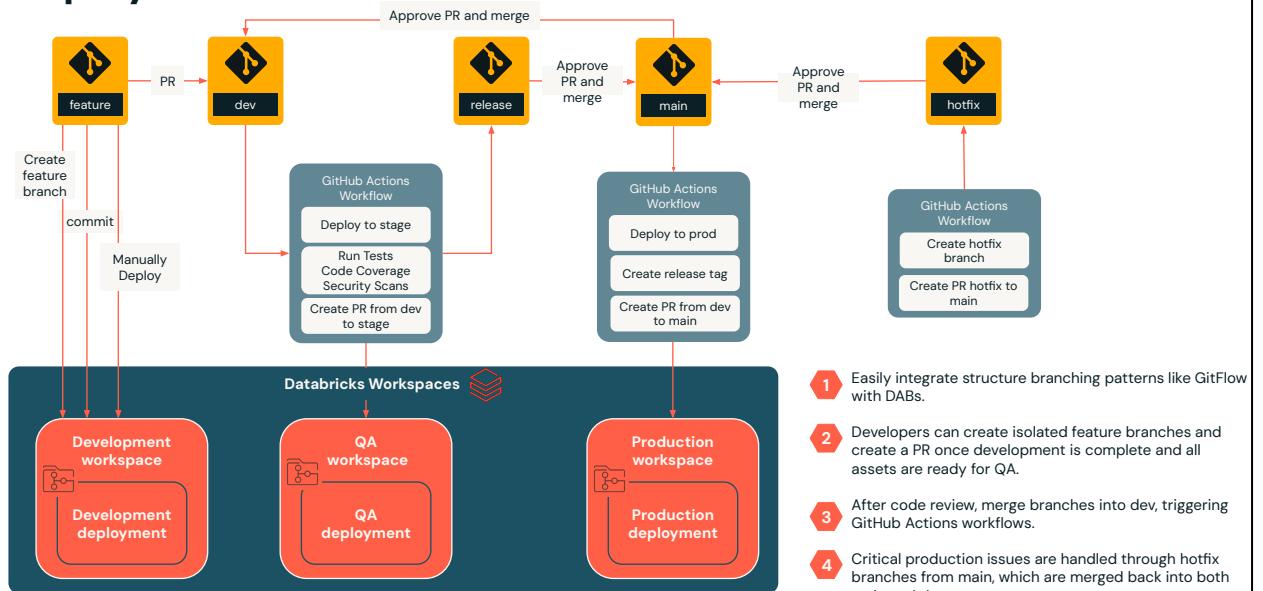


© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Deployment to QA and Production:

- After merging into develop, the next step involves triggering the GH Actions workflow for Drafting a release. This deploys the changes to the QA environment using DABs ,runs Tests, code coverage checks and security scans and finally creates the versioned release branch. It also creates a PR for the release branch to be merged into main.
- After the Release PR is closed, it triggers another GitHub actions workflow that deploys the bundle to PROD, creates a Release Tag and eventually, a PR for main branch to be merged into DEV.

Deployment Patterns with DABs and GitHub Actions



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](https://www.apache.org/).

Release Branching and Hotfixes:

- Finally, If a critical issue arises in production, we handle it with a hotfix branch, which is branched from **main** and then merged back into both **main** and **dev** after the fix is applied. This ensures that our production environment remains stable while allowing ongoing development to continue.

Reference:

<https://www.atlassian.com/git/tutorials/comparing-workflows/gitflow-workflow>

Overview of Git with Databricks

Definitions

Git is a free and open-source software framework designed to track changes in source code during software development



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

[READ DEFINITION OF GIT]

<CLICK>

Git Benefits: Version control enables tracking code changes, facilitating rollback and collaboration. Branching and merging allow multiple developers to work in parallel and integrate changes efficiently. A distributed workflow ensures each developer has a full local repository, enhancing flexibility and reliability. Git is optimized for high-performance handling of large projects, and its security features use cryptographic integrity checks to prevent data corruption.

<CLICK>

Git Tools & Services

1. **GitHub, GitLab, Bitbucket, Azure DevOps** – Cloud-based repositories with CI/CD, issue tracking, and team collaboration.
2. **Git CLI & GUI Clients (e.g., SourceTree, GitKraken, VS Code Git Integration)** – Provides different interfaces for managing repositories.
3. **CI/CD Integration** – Automated testing and deployment pipelines.
4. **Code Review & Collaboration** – Features like pull requests and merge approvals streamline teamwork.
5. **Security & Access Control** – Role-based permissions and audit logs enhance repository security.

1. Git operations, which we will discuss about on the next slide.
2. **Seamless Integration** - Users can leverage remote Git repos while developing code inside Databricks notebooks
3. **CI/CD Capabilities** - The repos REST API enables integration of data and AI projects into CI/CD pipelines, allowing users to automate Git workflows



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Always close with a thank you for participating in the course or watching this training video.